
ĐẠI HỌC PHENIKAA
TRƯỜNG CÔNG NGHỆ THÔNG TIN



BÁO CÁO ĐỒ ÁN CƠ SỞ
NHÓM 2: XÂY DỰNG ỨNG DỤNG QUẢN LÝ CHI TIÊU CÁ
NHÂN

Giảng viên hướng dẫn: TS.Vũ Quang Dũng

Sinh viên thực hiện: Đào Thu Trang

MSSV: 23010630

E-mail: 23010630@st.phenikaa-uni.edu.vn

Hà Nội, 2026.

Lời nói đầu

Trong cuộc sống hiện đại và bận rộn ngày nay, quản lý tài chính cá nhân đã trở thành một kỹ năng cực kỳ quan trọng, ảnh hưởng trực tiếp đến chất lượng cuộc sống của mỗi chúng ta, đặc biệt là những người trẻ. Thế nhưng, việc phải ngồi ghi chép chi tiêu thủ công mỗi ngày, tự tính toán số dư hay phân bổ ngân sách sao cho hợp lý thường tốn khá nhiều thời gian và rất dễ gây nản lòng.

Nhìn sang sự phát triển mạnh mẽ của công nghệ hiện nay, đặc biệt là Trí tuệ nhân tạo (AI) với các mô hình ngôn ngữ lớn, em nhận thấy công nghệ không còn chỉ là nơi để lưu trữ số liệu khô khan nữa. AI giờ đây đã có thể trò chuyện, phân tích và đưa ra những lời khuyên thông minh như một người trợ lý thực thụ. Xuất phát từ thực tế đó, em đã quyết định lựa chọn đề tài: **"Xây dựng ứng dụng quản lý chi tiêu cá nhân"** cho học phần Đồ án cơ sở của mình.

Ứng dụng được em xây dựng trên những công nghệ phổ biến hiện nay bao gồm React Native cho giao diện di động, Node.js/Express cho hệ thống máy chủ, MongoDB để lưu trữ dữ liệu và kết nối trực tiếp với API Google Gemini. Mục tiêu của sản phẩm là giúp việc ghi chép thu chi hàng ngày trở nên đơn giản hơn, đồng thời mang đến một người bạn đồng hành ảo giúp người dùng tối ưu hóa khoản tiền túi của mình một cách khoa học nhất.

Nội dung báo cáo Đồ án cơ sở của em được chia thành các phần cụ thể như sau:

- **Chương 1: Khảo sát bài toán và Xác định yêu cầu hệ thống** – Trình bày lý do em chọn đề tài, khảo sát thực trạng tiêu dùng và đưa ra các yêu cầu chi tiết về chức năng của ứng dụng.
- **Chương 2: Phân tích và Thiết kế hệ thống** – Cụ thể hóa kiến trúc phần mềm qua các sơ đồ UML (Use Case, sơ đồ lớp, sơ đồ tuần tự), cách thiết kế cơ sở dữ liệu và xây dựng các màn hình giao diện thực tế (UI/UX).
- **Chương 3: Một số thành phần khác của dự án** – Nhìn nhận về khía cạnh đạo đức nghề nghiệp, vấn đề bảo mật dữ liệu tài chính, những tác động tích cực của ứng dụng đối với xã hội và kế hoạch học tập thêm công nghệ mới của bản thân.
- **Chương 4: Kết luận và Đánh giá kết quả** – Tổng kết lại những gì em đã làm được, thẳng thắn đánh giá những điểm còn hạn chế của hệ thống như tốc độ phản hồi hay độ chính xác của AI, từ đó đưa ra hướng phát triển tiếp theo cho sản phẩm.

Vì đây là bước đầu em làm quen với việc xây dựng một hệ thống hoàn chỉnh và viết báo cáo khoa học, dù đã rất cố gắng và dành nhiều tâm huyết nhưng chắc chắn đồ án khó tránh khỏi những thiếu sót. Em rất mong sẽ nhận được những lời khuyên, nhận xét và góp ý từ thầy để em có thể hoàn thiện kỹ năng chuyên môn của mình tốt hơn trong tương lai.

Lời cảm ơn

Lời đầu tiên, em xin được bày tỏ lòng biết ơn sâu sắc và chân thành nhất tới thầy Vũ Quang Dũng – người đã trực tiếp hướng dẫn, định hướng đề tài và tận tình hướng dẫn em trong suốt quá trình thực hiện học phần Đồ án cơ sở này.

Đối với em, bước đầu làm quen với việc phát triển một hệ thống thực tế và viết báo cáo khoa học còn rất nhiều bỡ ngỡ và khó khăn. Nếu không có những buổi định hướng, những lời khuyên kịp thời và sự chỉ bảo tận tình của thầy, em chắc chắn sẽ mất rất nhiều thời gian để loay hoay tìm hướng đi cho đề tài. Những góp ý của thầy không chỉ giúp em tháo gỡ được các vướng mắc về mặt kỹ thuật mà còn giúp em hiểu ra thế nào là một tác phong làm việc nghiêm túc, chuẩn chỉ.

Dù đã dành nhiều thời gian và tâm huyết để hoàn thiện sản phẩm cũng như cuốn báo cáo này, nhưng do kiến thức và kinh nghiệm thực tế của bản thân em còn hạn chế, đồ án chắc chắn sẽ không tránh khỏi những thiếu sót. Em rất mong sẽ tiếp tục nhận được những lời nhận xét, đánh giá của thầy để em có cơ hội học hỏi và hoàn thiện bản thân hơn trong các học phần tiếp theo.

Cuối cùng, em xin kính chúc thầy luôn dồi dào sức khỏe, tràn đầy niềm vui, hạnh phúc và gặt hái được nhiều thành công hơn nữa trong sự nghiệp giáo dục cao quý của mình.

MỤC LỤC

Lời nói đầu	2
Lời cảm ơn	3
Chương 1. Giới thiệu tổng quan về đề tài	6
1.1 Đặt vấn đề	6
1.2 Các giải pháp đã có.....	6
1.3 Giải pháp đề xuất.....	7
Chương 2. Thiết kế và triển khai	8
2.1 Các yêu cầu chức năng.....	8
2.1.1 Nhóm chức năng hiển thị Tổng quan (Màn hình Trang chủ)	8
2.1.2 Nhóm chức năng Quản lý và Thêm mới giao dịch	8
2.1.3 Nhóm chức năng Thống kê và Phân tích xu hướng (Màn hình Biểu đồ)	9
2.1.4 Nhóm chức năng Lập kế hoạch và Quản lý Ngân sách	9
2.1.5 Nhóm chức năng Trợ lý AI và Tư vấn tài chính thông minh	10
2.2. Các yêu cầu phi chức năng	11
2.2.1 Yêu cầu về Hiệu năng.....	11
2.2.2 Yêu cầu về Bảo mật và Toàn vẹn dữ liệu	11
2.2.3 Yêu cầu về Tính khả dụng và Trải nghiệm người dùng	12
2.2.4 Yêu cầu về Tính tương thích và Khả năng di động.....	12
2.2.5 Yêu cầu về Tính tin cậy và Khả năng mở rộng.....	12
2.3. Các ràng buộc	13
2.3.1 Các ràng buộc về triển khai.....	13
2.3.2 Các ràng buộc kinh tế	13
2.3.3 Các ràng buộc về đạo đức và pháp lý	14
2.4. Mô hình hệ thống	15
2.4.1 Các kịch bản người dùng	15
2.4.2 Mô hình Use-case	18
2.4.3. Mô hình lớp và đối tượng.....	26
2.4.4 Các biểu đồ tuần tự	32
2.4.5 Các màn hình giao diện người dùng	36
Chương 3. Một số thành phần khác của dự án	44
3.1 Các vấn đề về đạo đức và làm việc chuyên nghiệp	44
3.1.1 Đạo đức trong quản lý và bảo mật dữ liệu người dùng	44
3.1.2 Đạo đức trong ứng dụng trí tuệ nhân tạo	44
3.1.3 Tác phong làm việc chuyên nghiệp và sở hữu trí tuệ	44
3.2 Tác động xã hội	45
3.2.1 Nâng cao nhận thức và năng lực tư duy tài chính cá nhân	45
3.2.2 Bình đẳng hóa cơ hội tiếp cận tư vấn chuyên gia nhờ công nghệ.....	45
3.2.3 Tác động tích cực đến tâm lý người tiêu dùng.....	45
3.3 Kế hoạch cho kiến thức mới và chiến lược học tập	45
3.3.1 Những kiến thức mới đã tiếp thu thành công qua dự án	45
3.3.2 Những hạn chế hiện tại của hệ thống cần khắc phục	46

Chương 4. Kết luận	47
4.1 Kết quả đạt được của dự án	47
4.1.1 Về mặt kiến thức và kỹ năng kỹ thuật.....	47
4.1.2 Về mặt sản phẩm công nghệ hoàn thiện.....	47
4.2 Hạn chế còn tồn tại.....	48
4.3 Hướng phát triển trong tương lai	48
Chương 5. Tài liệu tham khảo.....	50

Chương 1. Giới thiệu tổng quan về đề tài

1.1 Đặt vấn đề

Trong bối cảnh xã hội hiện đại, sự phát triển bùng nổ của các phương thức thanh toán điện tử (ví điện tử, mã QR, thẻ tín dụng) đã mang lại sự tiện lợi tối đa cho người tiêu dùng. Tuy nhiên, mặt trái của sự tiện lợi này là hiện tượng "chi tiêu không ma sát" (frictionless spending), khiến người dùng rất dễ đánh mất cảm giác về dòng tiền thực tế đang vơi đi.

Vấn đề cốt lõi mà đa số người trẻ, từ sinh viên đến người đi làm, đang gặp phải bao gồm:

- **Khó khăn trong việc theo dõi dòng tiền:** Các khoản chi lật vật hàng ngày thường bị bỏ quên, dẫn đến tình trạng "chưa hết tháng đã hết tiền" mà không rõ nguyên nhân.
- **Thiếu kế hoạch và kỷ luật tài chính:** Người dùng hiếm khi đặt ra các hạn mức chi tiêu cụ thể cho từng mục đích (ăn uống, giải trí, đi lại...), hoặc nếu có thì rất dễ vi phạm do thiếu cơ chế cảnh báo kịp thời.
- **Cảm giác quá tải khi quản lý thủ công:** Việc phải nhớ và nhập liệu từng khoản chi tiêu một cách cứng nhắc khiến người dùng nhanh chóng nản chí và từ bỏ việc quản lý tài chính chỉ sau một thời gian ngắn.

Từ những thực trạng trên, nhu cầu về một công cụ quản lý tài chính không chỉ dừng lại ở việc "ghi chép" thụ động, mà phải có khả năng "tương tác, nhắc nhở và phân tích" một cách thông minh đang trở nên cấp thiết hơn bao giờ hết.

1.2 Các giải pháp đã có

Hiện nay trên thị trường đã xuất hiện nhiều phương pháp và ứng dụng hỗ trợ quản lý chi tiêu. Tuy nhiên, mỗi giải pháp đều tồn tại những giới hạn nhất định:

- **Quản lý thủ công (Sổ tay, Excel):**
 - *Ưu điểm:* Hoàn toàn miễn phí, tùy biến cao.
 - *Hạn chế:* Rất tốn thời gian, dễ sai sót, dữ liệu bị phân mảnh và hoàn toàn không có khả năng tự động tính toán, nhắc nhở hay trích xuất biểu đồ phân tích nhanh chóng.
- **Các ứng dụng quản lý chi tiêu truyền thống (Ví dụ: Money Lover, Sổ Thu Chi MISA,...):**
 - *Ưu điểm:* Giao diện thân thiện, có sẵn các biểu đồ báo cáo và đồng bộ dữ liệu tốt.
 - *Hạn chế:* Phần lớn các ứng dụng này hoạt động như một "cuốn sổ điện tử" mang tính chất **thụ động**. Chúng chỉ cung cấp báo cáo *sau khi* giao dịch đã xảy ra (hậu kiểm). Bên cạnh đó, các danh mục chi tiêu thường bị "đóng gói" sẵn, thiếu tính linh hoạt cho các nhu cầu cá nhân hóa. Đặc biệt, chúng thiếu đi một "trợ lý ảo" đủ thông minh để đọc hiểu thói quen của người dùng và đưa ra lời khuyên ngăn chặn việc vung tay quá trán trước khi họ quyết định chi tiền.

1.3 Giải pháp đề xuất

Để khắc phục những hạn chế của các giải pháp hiện tại, đề tài này đề xuất xây dựng một **Ứng dụng/Web Quản lý chi tiêu thông minh tích hợp Trợ lý AI**. Ứng dụng được thiết kế nhằm chuyển đổi trạng thái của người dùng từ "ghi chép thụ động" sang "quản lý chủ động", với các tính năng cốt lõi bao gồm:

- **Cá nhân hóa hạng mục chi tiêu:** Cho phép người dùng tự do định nghĩa, thêm mới hoặc tinh chỉnh các hạng mục thu/chi phù hợp với lối sống và nhu cầu cá nhân thực tế (thay vì bị gò bó vào các khuôn mẫu có sẵn).
- **Lập kế hoạch và kiểm soát ngân sách linh hoạt:** Cung cấp công cụ thiết lập ngân sách theo tháng hoặc theo từng hạng mục cụ thể. Hệ thống sẽ có các mức độ cảnh báo trực quan (xanh, vàng, đỏ) dựa trên tiến độ tiêu dùng thời gian thực.
- **Trợ lý AI phân tích và can thiệp kịp thời:** Đây là điểm nhấn đột phá của ứng dụng. Trợ lý AI sẽ học hỏi từ dữ liệu chi tiêu lịch sử để:
 - Cảnh báo ngay lập tức khi phát hiện người dùng sắp vượt quá hạn mức ngân sách đã đề ra.
 - Đưa ra các báo cáo phân tích bằng ngôn ngữ tự nhiên, gợi ý cắt giảm các khoản chi không cần thiết để đảm bảo an toàn tài chính vào cuối tháng.

Chương 2. Thiết kế và triển khai

2.1 Các yêu cầu chức năng

Để đáp ứng trọn vẹn nhu cầu quản lý tài chính cá nhân một cách thông minh, trực quan cho học sinh, sinh viên và người đi làm, hệ thống được thiết kế và phân chia thành các nhóm chức năng cốt lõi sau:

2.1.1 Nhóm chức năng hiển thị Tổng quan (Màn hình Trang chủ)

Chức năng này đóng vai trò cung cấp cho người dùng một cái nhìn toàn cảnh, tức thời về trạng thái dòng tiền cá nhân trong tháng hiện tại.

- **Hiển thị dòng tiền tổng quát:** Hệ thống tự động tính toán và hiển thị rõ ràng ba chỉ số: Số dư tổng quan, Tổng thu nhập và Tổng chi tiêu của tháng.
- **Theo dõi tiến độ ngân sách:** Tích hợp thanh tiến trình (Progress Bar) trực quan hiển thị tỷ lệ phần trăm ngân sách đã sử dụng so với hạn mức tổng (ví dụ: đã tiêu dùng 30% ngân sách).
- **Bộ lọc nhanh danh sách:** Cho phép người dùng chuyển đổi linh hoạt giữa các tab "Tất cả", "Thu nhập" và "Chi tiêu" để lọc nhanh danh sách lịch sử.
- **Truy xuất lịch sử giao dịch gần đây:** Hiển thị danh sách các biến động số dư mới nhất, bao gồm đầy đủ thông tin: biểu tượng (icon) minh họa, tên hạng mục, số tiền (hiển thị màu xanh kèm dấu "+" đối với khoản thu và màu đỏ kèm dấu "-" đối với khoản chi), cùng ngày tháng phát sinh giao dịch.
- **Công cụ tương tác thời gian:** Cung cấp nút bấm tương tác nhanh (Chip) "Lọc theo ngày" giúp cá nhân hóa khoảng thời gian tìm kiếm dữ liệu.

2.1.2 Nhóm chức năng Quản lý và Thêm mới giao dịch

Chức năng này hỗ trợ người dùng ghi chép lại các hành vi tài chính phát sinh hàng ngày một cách nhanh chóng và tối giản.

- **Chuyển đổi chế độ ghi chép:** Cho phép người dùng linh hoạt chọn lựa giữa hai trạng thái là "+ Nhập" (ghi nhận giao dịch thực tế đã xảy ra) và "Kế hoạch" (ghi nhận các dự tính chi tiêu).
- **Phân loại dòng tiền:** Cung cấp hai nút lựa chọn lớn để phân định rõ ràng giữa "Chi tiêu" và "Thu nhập".
- **Cá nhân hóa thời gian giao dịch:** Tích hợp nút bấm chọn ngày hiển thị chi tiết theo định dạng thứ và ngày/tháng/năm để người dùng dễ dàng điều chỉnh thời gian phát sinh khoản chi.

- **Tối ưu hóa nhập liệu số tiền:** Bên cạnh ô nhập số tiền lớn trực quan, hệ thống cung cấp các nút "Chọn nhanh" mệnh giá phổ biến (50.000 đ, 100.000 đ, 200.000 đ, 500.000 đ) giúp giảm thiểu số lần chạm của người dùng.
- **Ghi chú bổ sung:** Hỗ trợ trường thông tin nhập văn bản tự do (không bắt buộc) để người dùng lưu lại bối cảnh chi tiết của giao dịch.

2.1.3 Nhóm chức năng Thống kê và Phân tích xu hướng (Màn hình Biểu đồ)

Chức năng này thực hiện nhiệm vụ tổng hợp và trực quan hóa các tập dữ liệu thô từ cơ sở dữ liệu thành thông tin hữu ích.

- **Đa dạng hóa kỳ báo cáo:** Hỗ trợ bộ lọc xem báo cáo chi tiết theo các mốc thời gian: "Ngày", "Tháng", hoặc "Năm".
- **Biểu đồ cột trực quan (Bar Chart):** Hệ thống tự động vẽ biểu đồ cột so sánh tương quan trực tiếp giữa Thu nhập (cột màu xanh) và Chi tiêu (cột màu đỏ) qua từng mốc thời gian cụ thể để người dùng nhận diện xu hướng tài chính.
- **Thống kê tỷ trọng danh mục chi tiêu:** Liệt kê danh sách các hạng mục đã chi dùng, tự động tính toán tỷ lệ phần trăm và sắp xếp theo thứ tự giảm dần, kết hợp thanh tiến trình màu sắc để người dùng biết rõ dòng tiền đang tập trung nhiều nhất ở đâu.
- **Cân đối kỳ tài chính:** Tính toán chỉ số "Số dư" thực tế còn lại của kỳ báo cáo để đánh giá mức độ thặng dư hoặc thâm hụt.

2.1.4 Nhóm chức năng Lập kế hoạch và Quản lý Ngân sách

Chức năng lập kế hoạch chi tiêu được thiết kế linh hoạt, chia làm hai giải pháp quản lý tài chính bổ trợ lẫn nhau, cho phép đáp ứng cả nhu cầu quản lý tổng quan lẫn chi tiết của người dùng:

- *Loại 1: Quản lý ngân sách tổng thể hệ thống (Tab "Cả tháng")*

Phương thức này hướng tới việc giúp người dùng thiết lập một bộ khung tài chính vững chắc ngay từ đầu tháng dựa trên nguyên lý cân bằng giữa dòng tiền vào, dòng tiền ra và quỹ tích lũy. Các trường thông tin cấu thành bao gồm:

- **Thu nhập dự kiến:** Số tiền người dùng ước tính sẽ nhận được trong tháng (Lương, trợ cấp, việc làm thêm...). Đây là mốc định vị để xây dựng các bài toán chi phối tài chính tiếp theo.
- **Hạn mức chi tiêu:** Ngưỡng tiền tối đa cho phép tiêu dùng trong tháng. Số tiền này bắt buộc phải nhỏ hơn thu nhập để đảm bảo an toàn tài chính.
- **Mục tiêu tiết kiệm:** Số tiền người dùng kỳ vọng giữ lại được để phục vụ các quỹ dự phòng hoặc đầu tư dài hạn.

- **Kế hoạch khác (Tùy chọn):** Không gian ghi chú tự do giúp người dùng lưu lại các cột mốc chỉ tiêu lớn cố định trong tháng (Ví dụ: mua xe cuối tháng, nộp học phí...).
- *Loại 2: Quy hoạch dòng tiền chi tiết (Tab "Theo hạng mục")*

Phương thức này áp dụng kỹ thuật quản lý tài chính bằng "phương pháp phong bì" (Envelope Method). Thay vì chỉ kiểm soát một con số tổng, người dùng chủ động phân rã dòng tiền của mình vào các "ngăn chứa" riêng biệt để kiểm soát triệt để thói quen tiêu dùng.

- **Cơ chế thiết lập:** Hệ thống liệt kê danh sách toàn bộ các hạng mục thiết yếu trong đời sống thường nhật bao gồm: *Ăn uống, Mua sắm, Di chuyển, Sức khỏe, Hóa đơn, Giải trí, Nhà ở, Du lịch, và Khác*. Người dùng có thể nhập hạn mức cụ thể cho từng mục dựa trên mức độ ưu tiên.
- **Tính linh hoạt cao:** Cho phép người dùng để trống các mục không có nhu cầu giới hạn và cung cấp tính năng **"Thêm hạng mục mới"** nhằm cá nhân hóa tối đa theo đặc thù lối sống của từng cá nhân.

2.1.5 Nhóm chức năng Trợ lý AI và Tư vấn tài chính thông minh

Đây là tính năng cốt lõi tạo nên sự đột phá của ứng dụng, chuyển đổi từ việc theo dõi thụ động sang tương tác chủ động dựa trên công nghệ Trí tuệ nhân tạo.

- **Đánh giá sức khỏe tài chính:** Hệ thống dựa trên các thuật toán tài chính và dữ liệu chi tiêu để chấm "Điểm sức khỏe tài chính" theo thang điểm 100, đồng thời gắn nhãn trạng thái tổng quan (ví dụ: Xuất sắc).
- **Phân tích thông minh tự động :** Tự động trích xuất các nhận định nhanh dưới dạng thẻ thông báo để cảnh báo người dùng (ví dụ: phát hiện hạng mục chiếm nhiều ngân sách nhất, thông báo số tiền còn lại trong quỹ).
- **Cảnh báo phân bổ chi tiêu trực quan:** Hiển thị danh sách các hạng mục đi kèm thanh tiến trình màu sắc thông minh; hệ thống sẽ tự động đổi sang màu cảnh báo (màu đỏ) đối với các danh mục đang có xu hướng chi tiêu vượt ngưỡng an toàn.
- **Hộp thoại tư vấn tài chính theo ngữ cảnh:** Cung cấp giao diện chat trực tiếp với Trợ lý AI. Người dùng có thể nhập bất kỳ câu hỏi bằng ngôn ngữ tự nhiên nào liên quan đến tình hình tài chính của bản thân.
- **Hệ thống câu hỏi gợi ý nhanh :** Tích hợp sẵn các nút gợi ý câu hỏi thực tế phổ biến như: *"Tôi đang chi tiêu như thế nào?", "Tôi có thể tiết kiệm thêm không?", "Đề xuất ngân sách tháng tới"*. Khi người dùng bấm vào, Trợ lý AI sẽ chủ động truy vấn cơ sở dữ liệu MongoDB để phân tích sâu và phản hồi ngay lập tức những giải pháp cá nhân hóa cho người dùng.

2.2. Các yêu cầu phi chức năng

Bên cạnh các tính năng tương tác trực tiếp, để ứng dụng vận hành một cách ổn định, mượt mà và an toàn trên thiết bị của người dùng, hệ thống của em phải đáp ứng đầy đủ các tiêu chuẩn kỹ thuật phi chức năng sau đây:

2.2.1 Yêu cầu về Hiệu năng

- **Tốc độ phản hồi API:** Đối với các tác vụ thêm, sửa, xóa giao dịch thông thường, thời gian từ lúc Frontend gọi hàm `fetch` trong `src/api/client.ts` qua REST API đến khi Backend Express.js và MongoDB Atlas xử lý thành công, trả về kết quả JSON phải đạt tốc độ **dưới 1 giây** ở điều kiện mạng tiêu chuẩn.
- **Thời gian phản hồi của Trợ lý AI:** Do các mô hình ngôn ngữ lớn cần thời gian xử lý ngữ cảnh lịch sử chi tiêu để đưa ra lời khuyên cá nhân hóa, thời gian phản hồi lý tưởng cần được kiểm soát **dưới 3 đến 5 giây**. Trong lúc hệ thống xử lý, Frontend bắt buộc phải hiển thị trạng thái chờ (Loading Indicator/Skeleton) để tránh cảm giác ứng dụng bị đóng băng/treo app.
- **Tốc độ phản hồi giao diện:** Ứng dụng di động cần duy trì tốc độ mượt mà ổn định ở mức **60 FPS** khi thực hiện các thao tác chuyển đổi qua lại giữa các tab (Trang chủ, Biểu đồ, Ngân sách, Trợ lý AI), cũng như khi cuộn danh sách lịch sử giao dịch dài mà không xảy ra hiện tượng giật, lag.

2.2.2 Yêu cầu về Bảo mật và Toàn vẹn dữ liệu

- **Bảo mật cấu hình hệ thống:** Toàn bộ thông tin nhạy cảm bao gồm chuỗi kết nối cơ sở dữ liệu (MongoDB Connection String) và API Key kết nối với Trợ lý AI (Claude/Gemini) phải được cô lập hoàn toàn ở phía Backend, lưu trữ trong file biến môi trường `.env`. Hệ thống tuyệt đối không để lộ các thông tin này ở mã nguồn Frontend để tránh bị khai thác trái phép.
- **Cơ chế ràng buộc dữ liệu (Data Validation):** Hệ thống thực hiện kiểm tra tính hợp lệ của dữ liệu ở cả 2 lớp:
 - *Frontend:* Ngăn chặn người dùng nhập ký tự chữ vào ô Số tiền, tự động định dạng tiền tệ thời gian thực để tránh nhầm lẫn.
 - *Backend:* Sử dụng Mongoose ODM Schema để kiểm tra nghiêm ngặt dữ liệu trước khi ghi vào MongoDB Atlas (Ví dụ: Số tiền giao dịch phải là số dương lớn hơn 0; các trường bắt buộc như Ngày, Hạng mục không được phép mang giá trị rỗng).

2.2.3 Yêu cầu về Tính khả dụng và Trải nghiệm người dùng

- **Thiết kế giao diện trực quan:** Vì đối tượng khách hàng mục tiêu kéo dài từ học sinh, sinh viên đến người đi làm, giao diện phải được tối giản hóa tối đa. Các thao tác nhập liệu phức tạp cần được chuyển đổi thành các nút bấm chọn nhanh (Quick Actions) như các mốc tiền 50.000 đ, 100.000 đ... giúp người dùng có thể hoàn thành việc ghi chép chi tiêu chỉ với 3 đến 4 lần chạm.
- **Tâm lý học màu sắc trong UI:** Giao diện ứng dụng phải tận dụng màu sắc để điều hướng tư duy người dùng một cách tinh tế. Màu xanh lá được gán cho Thu nhập và trạng thái Ngân sách an toàn; màu đỏ được gán cho Chi tiêu, số tiền âm hoặc trạng thái Ngân sách chạm ngưỡng báo động để tăng cường tính "cảnh tỉnh" và nhắc nhở kỷ luật tài chính.
- **Kích thước vùng tương tác:** Các ô bấm tương tác (như nút "Lọc theo ngày", các tab danh mục chi tiêu) được thiết kế theo dạng khối bo tròn góc tối đa (Chip/Pill) với kích thước tối thiểu đạt chuẩn thiết bị di động, đảm bảo người dùng có thể dễ dàng thao tác chính xác bằng một tay.

2.2.4 Yêu cầu về Tính tương thích và Khả năng di động

Đa nền tảng đồng nhất: Tận dụng tối đa sức mạnh của framework **React Native** và thư viện điều hướng **Expo Router v6**, ứng dụng phải đảm bảo hiển thị đồng nhất về mặt bố cục cấu trúc, màu sắc và chức năng khi biên dịch ra cả hai hệ điều hành di động phổ biến là **iOS** và **Android**.

- **Giao diện co giãn linh hoạt :** Ứng dụng phải tự động tính toán tỷ lệ kích thước màn hình thiết bị dựa trên Flexbox của React Native, đảm bảo hiển thị hoàn hảo trên nhiều dòng máy khác nhau, từ các thiết bị đời cũ màn hình nhỏ cho đến các dòng flagship màn hình lớn mà không bị tràn viền hay vỡ bố cục.

2.2.5 Yêu cầu về Tính tin cậy và Khả năng mở rộng

- **Tính sẵn sàng cao của Database:** Việc lưu trữ cơ sở dữ liệu trên nền tảng đám mây đám mây **MongoDB Atlas** giúp hệ thống của em thừa hưởng cơ chế sao lưu tự động (Automatic Backups) và phân tán dữ liệu, đảm bảo dữ liệu tài chính của người dùng không bị mất mát ngay cả khi có sự cố phần cứng xảy ra ở máy chủ vật lý.
- **Khả năng chịu tải đồng thời:** Nhờ vào kiến trúc hướng sự kiện bất đồng bộ của **Node.js** và **Express.js**, Backend của ứng dụng có khả năng tiếp nhận và xử lý đồng thời nhiều yêu cầu gửi về cùng một lúc, sẵn sàng đáp ứng tốt khi tệp người dùng của ứng dụng tăng trưởng trong tương lai.
- **Cơ chế xử lý ngoại lệ :** Khi gặp sự cố mất kết nối mạng hoặc API của bên thứ ba (Trợ lý AI) bị nghẽn, ứng dụng không được phép văng ra ngoài (Crash). Hệ thống phải thiết lập thời gian ngắt kết nối an toàn (Timeout) sau 5 giây và hiển thị thông báo lỗi thân thiện để hướng dẫn người dùng kiểm tra lại đường truyền.

2.3. Các ràng buộc

Trong quá trình nghiên cứu và phát triển hệ thống, bên cạnh việc thỏa mãn các yêu cầu về chức năng và chất lượng vận hành, em bắt buộc phải tuân thủ các giới hạn biên về mặt kỹ thuật, kinh tế và quy chuẩn đạo đức dưới đây. Đây là những điều kiện ràng buộc không thể thay đổi trong khuôn khổ thực hiện đồ án.

2.3.1 Các ràng buộc về triển khai

- **Ràng buộc về kiến trúc công nghệ:** Hệ thống bắt buộc phải được xây dựng dựa trên bộ framework đã định trước: Frontend sử dụng **TypeScript kết hợp React Native (Expo Router v6)**, Backend sử dụng **Node.js (Express.js)** và cơ sở dữ liệu **MongoDB Atlas**. Em không được tự ý thay đổi hoặc tích hợp các công nghệ không tương thích làm phá vỡ cấu trúc REST API hiện tại của file cấu hình `src/api/client.ts`.
- **Ràng buộc về môi trường vận hành:** Ứng dụng di động sau khi đóng gói bắt buộc phải chạy ổn định trên môi trường giả lập hoặc thiết bị thật thuộc hai hệ điều hành **Android** và **iOS** thông qua nền tảng Expo. Bản Backend phải được triển khai trên môi trường máy chủ cục bộ (Localhost) hoặc các dịch vụ đám mây miễn phí để đảm bảo Frontend có thể gọi lệnh `fetch` thành công qua môi trường mạng.
- **Ràng buộc về hạ tầng kết nối:** Do cơ sở dữ liệu MongoDB được lưu trữ trên Cloud (Atlas) và các mô hình Trợ lý AI hoạt động thông qua bên thứ ba (Web API), ứng dụng bắt buộc phải chạy trong môi trường thiết bị **có kết nối mạng Internet liên tục**. Hệ thống không hỗ trợ đồng bộ dữ liệu ngoại tuyến ở giai đoạn hiện tại.
- **Ràng buộc về thời gian:** Toàn bộ các công đoạn từ khảo sát, thiết kế, lập trình, kiểm thử cho đến hoàn thiện quyền báo cáo phải được thực hiện gói gọn và nộp đúng thời hạn theo lịch trình nghiêm ngặt của Nhà trường.

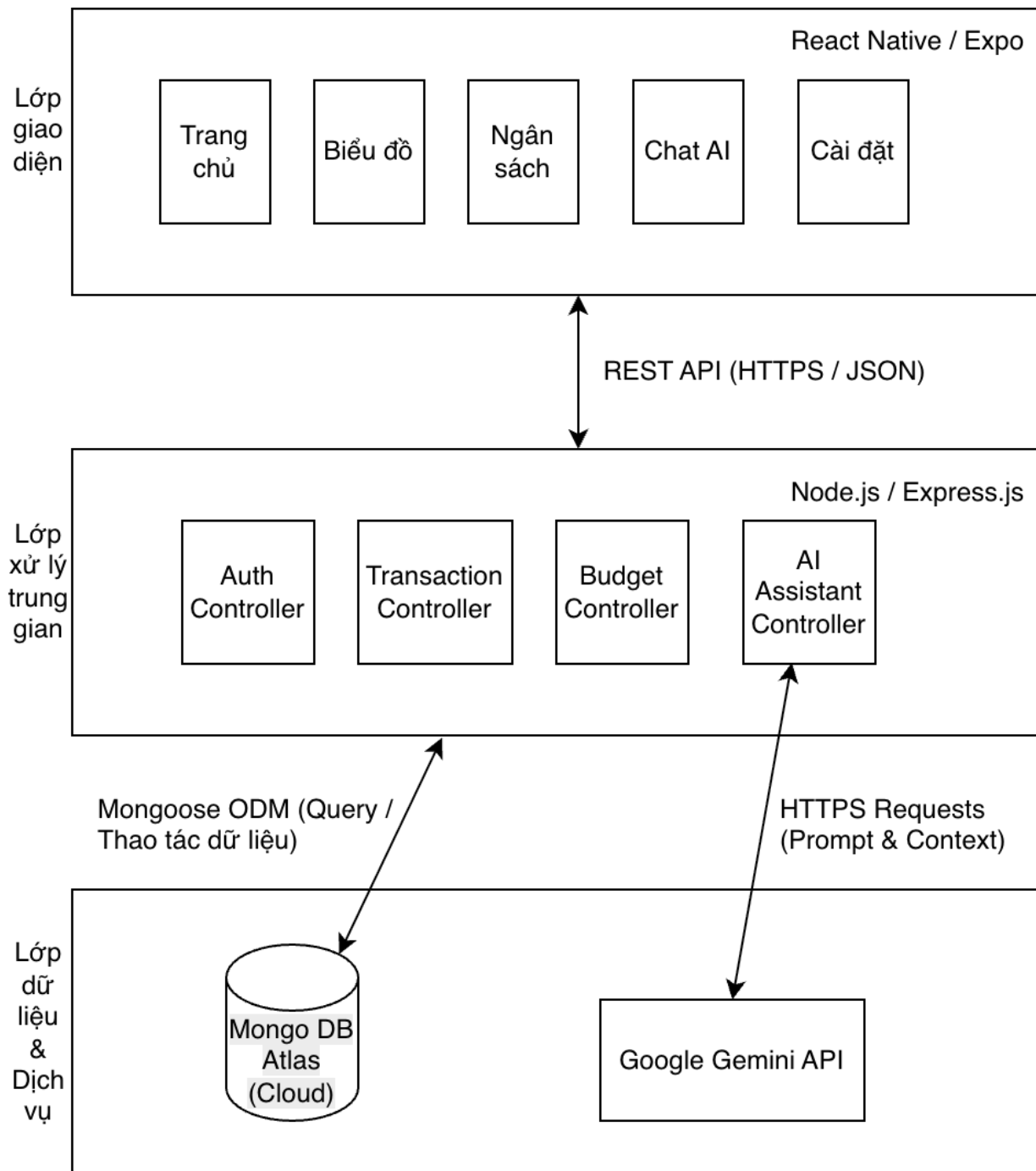
2.3.2 Các ràng buộc kinh tế

- **Hạn mức ngân sách phát triển:** Do đây là một đề tài nghiên cứu cấp sinh viên, tổng kinh phí đầu tư cho việc mua sắm trang thiết bị, bản quyền phần mềm và thuê máy chủ vận hành bằng **0 VNĐ**.
- **Giới hạn tài nguyên dịch vụ :** * Hệ thống quản trị cơ sở dữ liệu phải sử dụng gói cấu hình miễn phí **M0 Cluster** của MongoDB Atlas (giới hạn dung lượng lưu trữ tối đa 512MB). Điều này đòi hỏi em phải tối ưu hóa cấu trúc Schema dữ liệu sao cho gọn nhẹ nhất.
 - Việc tương tác với Trợ lý AI phải phụ thuộc hoàn toàn vào hạn mức tài khoản miễn phí (hoặc gói thử nghiệm giới hạn số lượt gọi mã API - Rate Limits theo phút/ngày) của các nhà cung cấp như OpenAI hay Google Gemini. Hệ thống có thể bị từ chối phản hồi nếu người dùng liên tục gửi câu hỏi vượt quá định mức miễn phí này.

2.3.3 Các ràng buộc về đạo đức và pháp lý

- **Tính bảo mật của dữ liệu tài chính nhạy cảm:** Thông tin về thu nhập, thói quen mua sắm và kế hoạch chi tiêu là những dữ liệu cực kỳ riêng tư của mỗi cá nhân. Hệ thống ràng buộc không được phép thực hiện bất kỳ hành vi thu thập ngầm, chia sẻ, mua bán hoặc rò rỉ dữ liệu của người dùng cho bên thứ ba. Toàn bộ dữ liệu lưu trữ trên MongoDB Atlas chỉ phục vụ duy nhất cho mục đích tính toán nội bộ của chính người dùng đó.
- **Trách nhiệm đối với lời khuyên từ AI (Disclaimer):** Do đặc điểm của các mô hình ngôn ngữ lớn (LLM) đôi khi có thể xảy ra hiện tượng "ảo tưởng dữ liệu" (Hallucination) hoặc đưa ra nhận định chưa hoàn toàn chính xác về mặt kinh tế, ứng dụng bắt buộc phải hiển thị **tuyên bố miễn trừ trách nhiệm pháp lý** ở màn hình Trợ lý AI. Hệ thống cần khẳng định: *"Mọi phản hồi, phân tích hay gợi ý chi tiêu của Trợ lý AI chỉ mang tính chất tham khảo học tập, không được coi là lời khuyên tư vấn tài chính hoặc đầu tư chuyên nghiệp có giá trị pháp lý"*. Điều này giúp bảo vệ quyền lợi của cả người phát triển lẫn người trải nghiệm ứng dụng.

2.4. Mô hình hệ thống



Sơ đồ 2.1. Sơ đồ khối kiến trúc 3 lớp của hệ thống

2.4.1 Các kịch bản người dùng

Để làm rõ luồng tương tác giữa Người dùng và Hệ thống qua từng màn hình chức năng cụ thể, phần này trình bày kịch bản chi tiết của 3 nhóm chức năng cốt lõi.

- Kịch bản 1 : Đăng nhập và Xem tổng quan tài chính

Kịch bản này mô tả cách người dùng truy cập hệ thống và theo dõi biến động dòng tiền tức thời.

- **Tác nhân (Actor):** Người dùng.
- **Mục đích:** Truy cập ứng dụng để xem số dư, tổng thu chi và các biểu đồ phân tích xu hướng.
- **Luồng sự kiện chính (Basic Flow):**
 1. Người dùng tiến hành đăng nhập vào ứng dụng bằng tài khoản cá nhân.
 2. Hệ thống xác thực thông tin thành công và tự động điều hướng người dùng đến màn hình **Trang chủ** đầu tiên.
 3. Tại Trang chủ, hệ thống truy vấn và hiển thị: Số dư tổng quan, Tổng thu nhập, Tổng chi tiêu, thanh tiến trình ngân sách tháng cùng danh sách giao dịch gần đây.
 4. Người dùng nhấn vào Tab **Biểu đồ** trên thanh điều hướng phía dưới.
 5. Hệ thống chuyển đổi sang giao diện Biểu đồ chi tiêu, hiển thị trực quan các cột Thu/Chi, tỷ trọng các hạng mục và chỉ số cân đối kỳ tài chính.
- Kịch bản 2 : Ghi chép giao dịch và Thiết lập kế hoạch Ngân sách

Kịch bản này mô tả toàn bộ nghiệp vụ thêm mới dòng tiền và các kịch bản xử lý thông minh của hệ thống khi người dùng chi tiêu vượt ngưỡng an toàn.

- **Tác nhân (Actor):** Người dùng.
- **Mục đích:** Ghi nhận khoản thu/chi thực tế hoặc xây dựng hạn mức ngân sách tháng để kiểm soát dòng tiền.
- **Luồng sự kiện chính :**
 1. Người dùng nhấn vào nút chức năng thêm mới ở thanh điều hướng.
 2. Hệ thống hiển thị giao diện **Thêm giao dịch**. Tại đây, người dùng có thể chọn ghi chép nhanh giao dịch thực tế hoặc chọn chuyển sang tab **Kế hoạch** để thiết lập tài chính.
 3. *Nếu chọn Lập kế hoạch cả tháng:* Người dùng nhập các thông tin: Thu nhập dự kiến, Hạn mức chi tiêu và Mục tiêu tiết kiệm.
 4. *Nếu chọn Kế hoạch theo hạng mục:* Người dùng thực hiện phân bổ dòng tiền chi tiết cho từng danh mục riêng biệt như Ăn uống, Nhà ở, Di chuyển...
 5. Sau khi người dùng hoàn tất nhập liệu và nhấn lưu, hệ thống thực hiện kiểm tra các điều kiện ràng buộc ngân sách và ghi dữ liệu vào MongoDB.
- **Các luồng sự kiện thay thế và ngoại lệ :**
 - **Trường hợp chi tiêu chạm ngưỡng cảnh báo (Tiêu quá 80% - 90% ngân sách danh mục/tháng):** Ngay tại giao diện nhập số tiền chi, hệ thống sẽ tự động tính toán tỷ lệ, chuyển đổi màu sắc các thành phần giao diện chủ đạo sang **màu Vàng hoặc màu Cam** để đưa ra cảnh báo thị giác sớm, nhắc nhở người dùng cân nhắc trước khi tiêu dùng.

- **Trường hợp chi tiêu vượt ngưỡng giới hạn (Tiêu quá 100% ngân sách):** Khi người dùng cố tình nhập một khoản chi khiến tổng chi tiêu vượt quá 100% hạn mức cho phép, hệ thống sẽ gửi một thông báo nhắc nhở (Pop-up alert): *"Khoản chi này sẽ làm bạn vượt quá ngân sách tháng. Bạn có chắc chắn muốn lưu giao dịch này không?"*.
 - Nếu người dùng chọn hủy: Hệ thống giữ nguyên màn hình nhập để chỉnh sửa.
 - Nếu người dùng xác nhận lưu: Hệ thống chấp nhận ghi nhận, đồng thời tại màn hình hiển thị lịch sử, giao dịch này sẽ được **bao bọc bởi một khung màu Đỏ rực** nhằm nhấn mạnh việc vi phạm kỷ luật tài chính.

- Kịch bản 3: Tương tác và Tham vấn Trợ lý AI

Kịch bản này mô tả cách người dùng tiếp cận các phân tích chuyên sâu được cá nhân hóa bởi công nghệ Trí tuệ nhân tạo.

- **Tác nhân (Actor):** Người dùng, Hệ thống AI.
- **Mục đích:** Xem đánh giá sức khỏe tài chính và trò chuyện bằng ngôn ngữ tự nhiên với Trợ lý ảo.
- **Luồng sự kiện chính :**
 1. Người dùng chọn tab **Trợ lý AI** trên thanh điều hướng.
 2. Hệ thống tổng hợp dữ liệu từ cơ sở dữ liệu và hiển thị giao diện phân cấp thông tin theo thứ tự từ trên xuống dưới:
 - Khung hiển thị **Điểm tài chính cá nhân** dựa trên mức độ hoàn thiện mục tiêu tiết kiệm.
 - Khung **Phân tích thông minh ngắn (AI Insights)** trích xuất trực tiếp xu hướng tiêu dùng chính (Ví dụ: *"Hạng mục Nhà ở đang chiếm tỷ trọng lớn nhất với 60% tổng chi tiêu của bạn"*).
 - Khung tổng quan biểu diễn biểu đồ **tổng % số tiền đã tiêu** phân bổ theo các khu vực, lĩnh vực.
 - Khung giao diện **Hỏi đáp trực tiếp với Trợ lý AI** ở phía cuối màn hình.
 3. Người dùng nhập nội dung câu hỏi vào ô chat hoặc chọn các câu hỏi gợi ý nhanh.
 4. Hệ thống AI tiếp nhận ngữ cảnh dữ liệu, xử lý ngôn ngữ và phản hồi các giải pháp, lời khuyên tài chính phù hợp cho người dùng.

- Kịch bản 4: Quản lý Cài đặt hệ thống

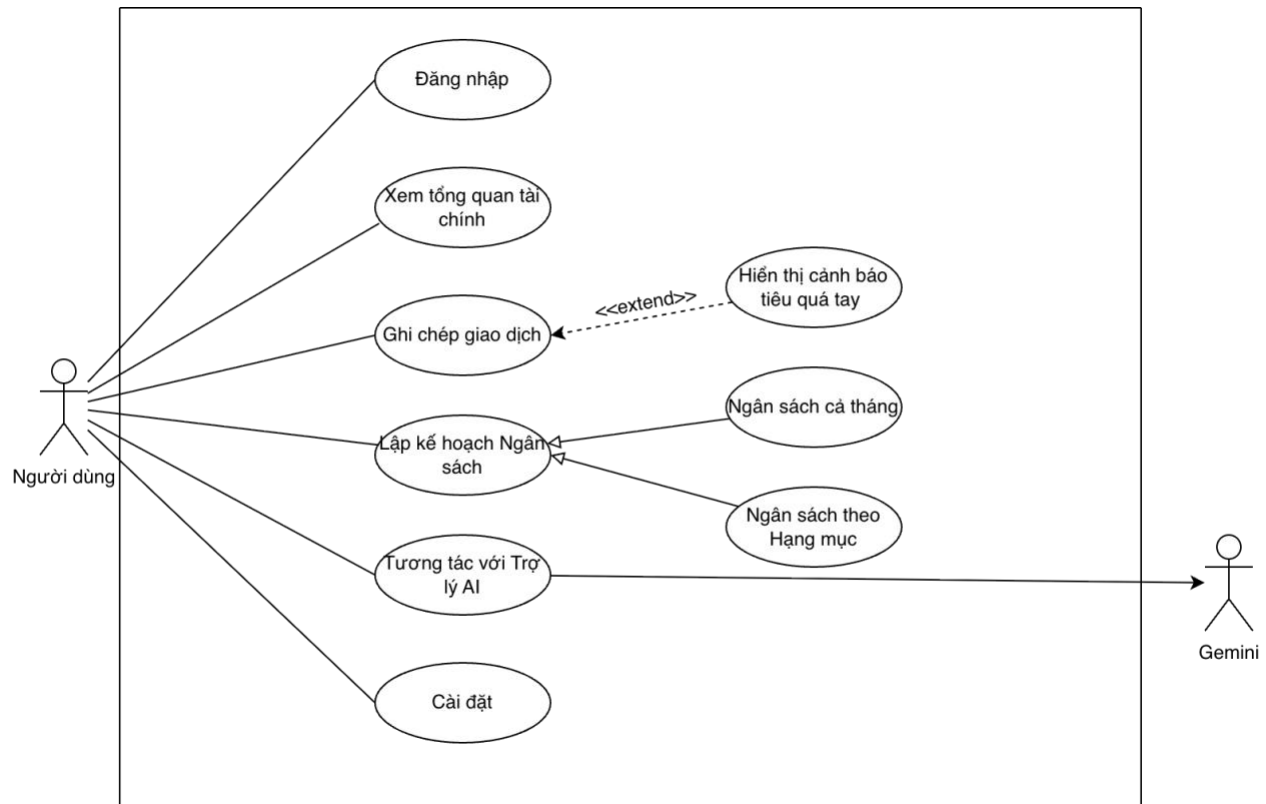
- **Tác nhân (Actor):** Người dùng.
- **Mục đích:** Tra cứu thông tin cá nhân và kiểm tra các thông số kỹ thuật của phần mềm.
- **Luồng sự kiện chính (Basic Flow):**
 1. Người dùng nhấn chọn tab **Cài đặt**.

2. Hệ thống hiển thị hai phân vùng thông tin lớn:

- *Thông tin cá nhân*: Cho phép quản lý tên, email, ảnh đại diện và trạng thái tài khoản.
- *Thông tin ứng dụng*: Hiển thị phiên bản phần mềm (Version), điều khoản sử dụng và chính sách bảo mật dữ liệu tài chính.

2.4.2 Mô hình Use-case

Nhằm mục đích hệ thống hóa toàn bộ các chức năng và mối quan hệ giữa người dùng với các thực thể trong ứng dụng Quản lý tài chính cá nhân, sơ đồ Use-case tổng quan được thiết kế như sau:



Sơ đồ 2.2. Sơ đồ Use-case tổng quan

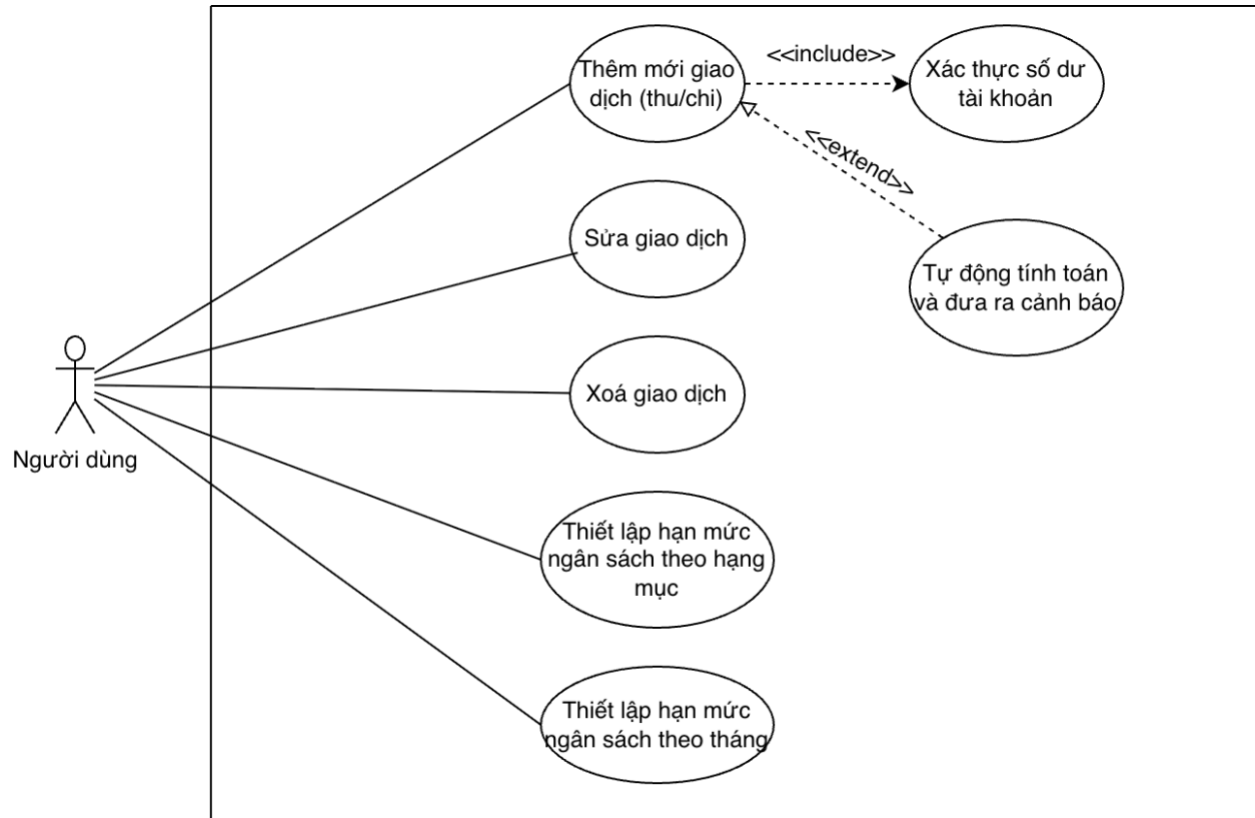
Để làm rõ hơn các quy tắc ràng buộc kỹ thuật, mối quan hệ bao hàm (<<include>>), mối quan hệ mở rộng (<<extend>>) cũng như các điều kiện xử lý dữ liệu sâu bên trong, hệ thống được phân rã thành các phân hệ chức năng chi tiết đi kèm bảng đặc tả cụ thể dưới đây:

2.4.2.1 Module Quản lý Giao dịch và Ngân sách thông minh

Module này đảm nhận vai trò xương sống của ứng dụng, chịu trách nhiệm xử lý toàn bộ các tác vụ biến động dòng tiền (Thu nhập, Chi tiêu) và thiết lập các hạn mức tài chính định kỳ của người dùng. Để tối ưu hóa trải nghiệm, module được tích hợp thêm cơ chế tự động tính toán

và đưa ra các cảnh báo trực quan bằng màu sắc khi người dùng có xu hướng chi tiêu vượt ngưỡng.

Sơ đồ Use-case phân rã chi tiết của module Giao dịch và Ngân sách được mô tả như sau:



Sơ đồ 2.3. Sơ đồ Use-case chức năng Giao dịch và Ngân sách

Dựa trên các mối quan hệ được thiết lập trong sơ đồ phân rã, cấu trúc kỹ thuật và luồng sự kiện chi tiết của từng chức năng được đặc tả cụ thể qua các bảng sau:

Bảng 1: Đặc tả Use-case Thêm mới giao dịch

Thành phần	Nội dung mô tả chi tiết
Mã Use-case	UC-01
Tên Use-case	Thêm mới giao dịch (thí/chi)
Tác nhân	Người dùng
Mục đích	Cho phép người dùng tạo mới một bản ghi thu nhập hoặc chi tiêu vào hệ thống để quản lý dòng tiền.
Điều kiện tiên quyết	Người dùng đã đăng nhập thành công và thiết bị kết nối mạng ổn định.

Luồng sự kiện chính	1. Người dùng nhấn chọn biểu tượng Thêm mới giao dịch. 2. Hệ thống hiển thị biểu mẫu nhập thông tin giao dịch. 3. Người dùng nhập số tiền, chọn phân loại (Thu hoặc Chi), chọn Hạng mục (Ăn uống, Mua sắm...) và chọn Ngày thực hiện. 4. Người dùng nhấn nút "Lưu giao dịch". 5. Hệ thống gọi thực hiện tự động UC-02: Xác thực số dư tài khoản (Mối quan hệ <<include>>). 6. Hệ thống kiểm tra điều kiện ngân sách và tự động kích hoạt UC-03: Tự động tính toán và đưa ra cảnh báo nếu vi phạm hạn mức (Mối quan hệ <<extend>>). 7. Hệ thống ghi dữ liệu mới vào MongoDB Atlas. 8. Hệ thống thông báo thành công và điều hướng người dùng về Trang chủ.
Luồng ngoại lệ	Khi dữ liệu đầu vào lỗi hoặc sai định dạng hệ thống hiển thị thông báo lỗi "Vui lòng nhập đầy đủ số tiền và chọn hạng mục" và giữ nguyên giao diện biểu mẫu để người dùng sửa.
Điều kiện hậu quyết	Bản ghi giao dịch được lưu trữ vĩnh viễn, số dư tổng và các biểu đồ phân tích được cập nhật số liệu mới tức thời.

Bảng 2: Đặc tả Use-case Xác thực số dư tài khoản

Thành phần	Nội dung mô tả chi tiết
Mã Use-case	UC-02
Tên Use-case	Xác thực số dư tài khoản

Tác nhân	Hệ thống (Mongo DB)
Mục đích	Tự động tính toán và cập nhật số dư khả dụng của tài khoản ngay khi có biến động giao dịch.
Điều kiện tiên quyết	Được kích hoạt bắt buộc bởi các hành động Thêm (UC-01), Sửa (UC-04), hoặc Xóa (UC-05).
Luồng sự kiện chính	1. Hệ thống tiếp nhận thông tin số tiền và phân loại (Thu/Chi) từ giao dịch vừa tác động. 2. Hệ thống lấy giá trị số dư hiện tại trong ví của User. 3. Hệ thống thực hiện phép tính: Số dư mới = Số dư cũ + Thu nhập HOẶC Số dư mới = Số dư cũ - Chi tiêu. 4. Hệ thống cập nhật lại trường balance của người dùng trong DB và phản hồi trạng thái hợp lệ về cho chức năng gọi.
Điều kiện hậu quyết	Số tổng tài sản hiển thị ngoài trang chủ được làm mới khớp với các giao dịch thực tế.

Bảng 3: Đặc tả Use-case Tự động tính toán và đưa ra cảnh báo

Thành phần	Nội dung mô tả chi tiết
Mã Use-case	UC-03
Tên Use-case	Tự động tính toán và đưa ra cảnh báo quá tay
Tác nhân	Hệ thống xử lý logic tại Backend (Node.js)
Mục đích	Giám sát ngưỡng chi tiêu của người dùng so với hạn mức ngân sách đã lập để đưa ra cảnh báo trực quan bằng màu sắc.
Điều kiện tiên quyết	Được kích hoạt mở rộng từ chức năng Thêm mới giao dịch (UC-01) khi giao dịch đó thuộc phân loại "Chi tiêu".
Luồng sự kiện chính	1. Hệ thống lấy tổng số tiền đã chi trong tháng (hoặc trong hạng mục đó) cộng với số tiền của giao dịch hiện tại.

	2. Hệ thống so sánh tổng chỉ tiêu này với hạn mức ngân sách người dùng đặt trước đó. 3. Trường hợp hạn mức đạt từ 80% đến dưới 100%: Hệ thống gửi tín hiệu yêu cầu Frontend đổi màu sắc thanh trạng thái sang màu Cam. 4. Trường hợp hạn mức vượt quá 100%: Hệ thống kích hoạt một Pop-up thông báo khẩn cấp "Bạn đã tiêu vượt quá ngân sách đặt ra!". Nếu người dùng nhấn đồng ý tiếp tục, hệ thống sẽ bao bọc giao dịch này bằng khung viền Đỏ trong danh sách lịch sử.
Điều kiện hậu quyết	Người dùng nhận biết được trạng thái an toàn tài chính thông qua dấu hiệu thị giác (Màu Cam/Màu Đỏ).

Bảng 4: Đặc tả Use-case Sửa giao dịch

Thành phần	Nội dung mô tả chi tiết
Mã Use-case	UC-04
Tên Use-case	Sửa giao dịch
Tác nhân	Người dùng
Mục đích	Cho phép người dùng chỉnh sửa thông tin của một giao dịch cũ đã ghi chép sai.
Luồng sự kiện chính	1. Người dùng chọn một giao dịch cụ thể trong danh sách lịch sử và nhấn "Chỉnh sửa". 2. Hệ thống hiển thị lại toàn bộ thông tin cũ lên biểu mẫu. 3. Người dùng thay đổi thông tin (Số tiền, ngày, hạng mục...) và nhấn "Cập nhật". 4. Hệ thống gọi thực hiện tự động UC-02 để tính toán lại số dư dựa trên chênh lệch cũ - mới. 5. Hệ thống lưu đè dữ liệu mới vào DB và thông báo thành công.

Bảng 5: Đặc tả Use-case Xóa giao dịch

Thành phần	Nội dung mô tả chi tiết
Mã Use-case	UC-05
Tên Use-case	Xóa giao dịch
Tác nhân	Người dùng
Mục đích	Hủy bỏ một bản ghi giao dịch không còn giá trị hoặc do nhập nhầm.
Luồng sự kiện chính	1. Người dùng chọn giao dịch cần xóa và nhấn biểu tượng "Thùng rác". 2. Hệ thống hiển thị hộp thoại cảnh báo: "Bạn có chắc chắn muốn xóa giao dịch này?". 3. Người dùng nhấn "Xác nhận xóa". 4. Hệ thống gọi thực hiện tự động UC-02 để hoàn tác lại số tiền đã xóa vào số dư tổng. 5. Hệ thống xóa vĩnh viễn bản ghi khỏi cơ sở dữ liệu.

Bảng 6: Đặc tả Use-case Thiết lập hạn mức ngân sách theo hạng mục

Thành phần	Nội dung mô tả chi tiết
Mã Use-case	UC-06
Tên Use-case	Thiết lập hạn mức ngân sách theo hạng mục
Tác nhân	Người dùng
Mục đích	Giới hạn số tiền tối đa được phép chi tiêu cho một hoạt động cụ thể (Ví dụ: Chi tiêu tối đa 2 triệu/tháng cho Ăn uống).
Luồng sự kiện chính	1. Người dùng truy cập Tab Ngân sách, chọn phân hệ "Theo hạng mục". 2. Người dùng chọn một hạng mục cụ thể trong danh sách thả xuống. 3. Người dùng nhập số tiền giới hạn mong muốn và bấm "Lưu cấu hình". 4. Hệ thống lưu thông tin vào bảng dữ liệu ngân sách hạng mục và áp dụng làm mốc tính toán cho UC-03.

Bảng 7: Đặc tả Use-case Thiết lập hạn mức ngân sách theo tháng

Thành phần	Nội dung mô tả chi tiết
Mã Use-case	UC-07
Tên Use-case	Thiết lập hạn mức ngân sách theo tháng
Tác nhân	Người dùng
Mục đích	Khóa trần tổng chi tiêu của toàn bộ ứng dụng trong một tháng dương lịch (Ví dụ: Tổng chi cả tháng không quá 5 triệu).
Luồng sự kiện chính	1. Người dùng chọn phân hệ "Ngân sách cả tháng". 2. Người dùng nhập số tiền tổng hạn mức cho cả tháng và nhấn "Xác nhận". 3. Hệ thống tạo mới/Cập nhật cấu hình ngân sách tháng của User trong DB và hiển thị tiến trình sử dụng tài chính dạng phần trăm ngoài màn hình chính.

2.4.2.2 Module Trợ lý tài chính AI và Chức năng hệ thống

Bên cạnh các tính năng quản lý truyền thống, chức năng Trợ lý tài chính AI đóng vai trò là điểm nhấn thông minh của ứng dụng. Chức năng này chịu trách nhiệm kết nối dữ liệu thô từ cơ sở dữ liệu, đóng gói ngữ cảnh để giao tiếp với trí tuệ nhân tạo, từ đó trả về lời khuyên tài chính cá nhân hóa. Song song đó là các tính năng nền tảng như Đăng nhập để bảo mật thông tin.

Các luồng sự kiện và điều kiện ngoại lệ của chức năng này được tiến hành đặc tả chi tiết qua các bảng dưới đây:

Bảng 8: Đặc tả Use-case Tương tác với trợ lý AI

Thành phần	Nội dung mô tả chi tiết
Mã Use-case	UC-08
Tên Use-case	Tương tác với trợ lý AI
Tác nhân	Người dùng, Hệ thống AI (Gemini)
Mục đích	Cung cấp điểm số đánh giá tài chính cá nhân và giải đáp, đưa ra lời khuyên chi tiêu thông minh bằng ngôn ngữ tự nhiên thông qua giao diện Chatbot.

Điều kiện tiên quyết	Thiết bị người dùng giữ kết nối Internet liên tục để gọi API.
Luồng sự kiện chính	1. Người dùng bấm chọn Tab Trợ lý AI trên thanh điều hướng. 2. Hệ thống tự động hiển thị Điểm tài chính và phân tích phần trăm chỉ tiêu lớn nhất dựa trên dữ liệu tháng hiện tại. 3. Người dùng nhập nội dung câu hỏi (Ví dụ: "Tháng này mình tiêu thế nào?") và nhấn "Gửi". 4. Lớp Backend (Node.js) tiếp nhận, truy vấn dữ liệu chỉ tiêu thô của người dùng từ MongoDB Atlas. 5. Backend xử lý cấu trúc dữ liệu, kết hợp với câu hỏi của người dùng để đóng gói thành một ngữ cảnh đầu vào (Prompt Context) bảo mật và gửi lệnh gọi sang Google Gemini API thông qua Axios. 6. Gemini API phân tích dữ liệu và trả lại chuỗi văn bản chứa lời khuyên chuyên gia kỹ lưỡng. 7. Hệ thống tiếp nhận kết quả trả về và hiển thị trực quan dưới dạng bong bóng chat trên giao diện người dùng.
Luồng ngoại lệ	Khi kết nối mạng bị ngắt hoặc API quá tải hệ thống hiển thị thông báo: "Kết nối đến Trợ lý AI tạm thời bị gián đoạn. Vui lòng kiểm tra lại đường truyền mạng hoặc thử lại sau" và không gây treo ứng dụng.

Bảng 9: Đặc tả Use-case Đăng nhập hệ thống

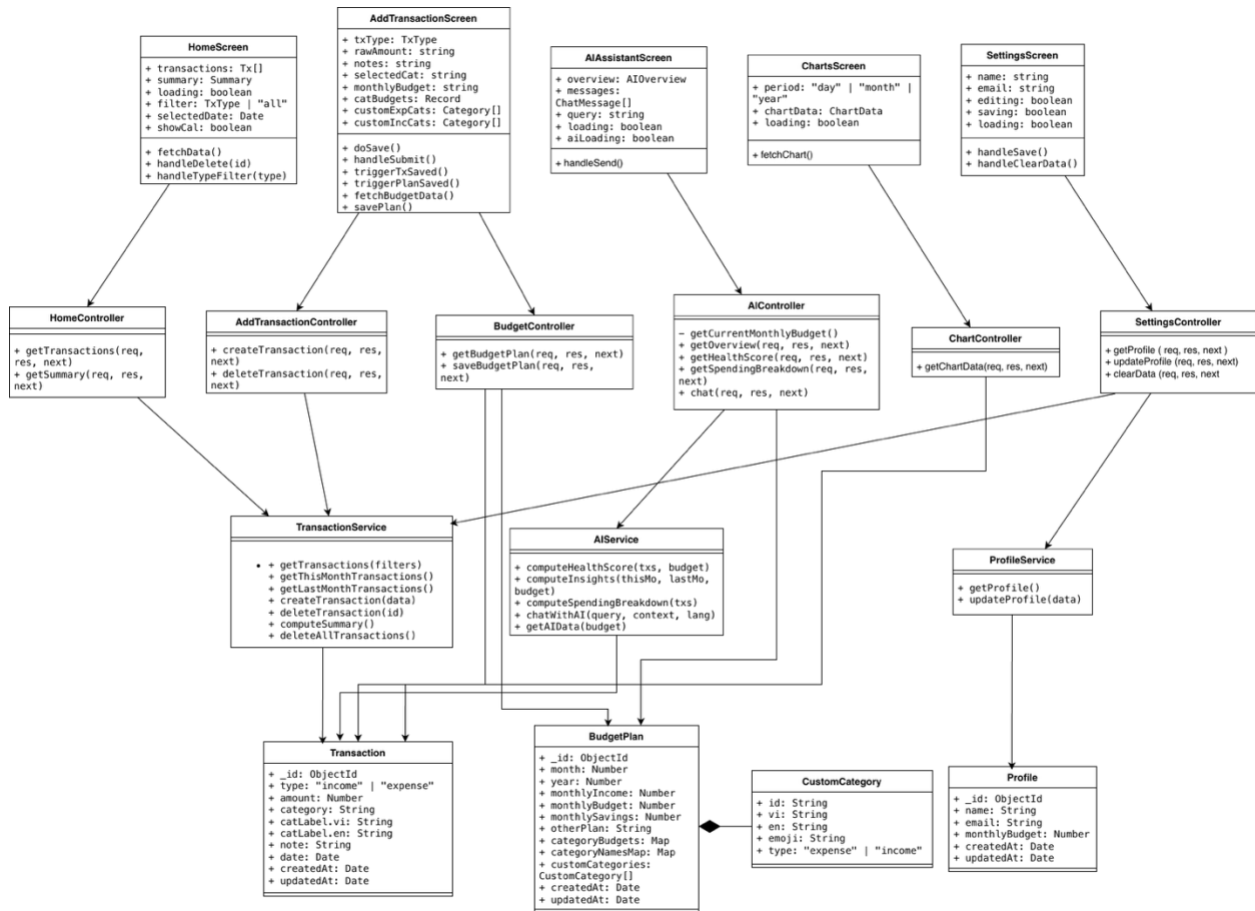
Thành phần	Nội dung mô tả chi tiết
Mã Use-case	UC-09
Tên Use-case	Đăng nhập

Tác nhân	Người dùng
Mục đích	Xác thực danh tính người dùng để bảo mật thông tin tài chính cá nhân trước khi cho phép truy cập các tính năng sâu của ứng dụng.
Luồng sự kiện chính	1. Người dùng mở ứng dụng, hệ thống chuyển đến màn hình Đăng nhập. 2. Người dùng nhập Email và Mật khẩu, nhấn "Đăng nhập". 3. Backend băm mật khẩu, đối chiếu thông tin trong cơ sở dữ liệu MongoDB. 4. Nếu trùng khớp thông tin, hệ thống cấp mã Token xác thực, lưu trạng thái phiên đăng nhập và mở khóa chuyển người dùng vào màn hình Trang chủ.

2.4.3. Mô hình lớp và đối tượng

Nhằm cụ thể hóa cấu trúc dữ liệu cấu hình tại lớp Cơ sở dữ liệu và các hàm xử lý logic tại lớp xử lý trung gian (Backend Server), mô hình lớp (Class Diagram) được xây dựng. Sơ đồ này mô tả chi tiết các thực thể dữ liệu, các thuộc tính ràng buộc kèm kiểu dữ liệu, các phương thức hoạt động cùng mối quan hệ tương tác (bậc tham chiếu 1 - n) giữa các thành phần bên trong ứng dụng Quản lý tài chính cá nhân.

Dưới đây là sơ đồ lớp chi tiết của hệ thống:



Sơ đồ 2.4. Sơ đồ Class Diagram tổng thể của ứng dụng

Để làm rõ kiến trúc nội tại và cơ chế giao tiếp giữa các thành phần, cấu trúc chi tiết của từng lớp đối tượng được đặc tả theo các phân tầng kỹ thuật cụ thể dưới đây:

2.4.3.1 Lớp Dữ liệu và Cấu trúc lược đồ

Phân tầng này định nghĩa cấu trúc của các tài liệu (Documents) được lưu trữ trong MongoDB thông qua Mongoose ODM, đảm bảo tính toàn vẹn dữ liệu cho hệ thống.

1. Lớp Transaction (Quản lý Giao dịch)

- **Ý nghĩa:** Lưu trữ chi tiết các biến động số tiền thu nhập và chi tiêu.

- **Các thuộc tính:**

- + `_id`: ObjectId (Khóa chính tự sinh)
- + `type`: String (Phân loại: income hoặc expense)
- + `amount`: Number (Số tiền giao dịch)
- + `category`: String (Mã định danh biểu tượng Emoji)
- + `catLabel.vi`: String (Tên hạng mục tiếng Việt)
- + `catLabel.en`: String (Tên hạng mục tiếng Anh)
- + `note`: String (Ghi chú chi tiết)
- + `date`: Date (Ngày phát sinh giao dịch)
- + `createdAt`: Date / + `updatedAt`: Date (Thời gian tạo và cập nhật)

2. Lớp Profile (Thông tin Người dùng)

- **Ý nghĩa:** Lưu trữ thông tin cá nhân và cấu hình tài chính cơ bản.
- **Các thuộc tính:**
 - + `_id`: `ObjectId` (Khóa chính)
 - + `name`: `String` / + `email`: `String` (Tên và Email định danh)
 - + `monthlyBudget`: `Number` (Hạn mức chi tiêu tổng mặc định của tháng)
 - + `createdAt`: `Date` / + `updatedAt`: `Date` (Thời gian hệ thống)

3. Lớp BudgetPlan (Kế hoạch Ngân sách)

- **Ý nghĩa:** Lưu trữ cấu hình kế hoạch ngân sách chuyên sâu theo từng tháng/năm, bao gồm cả các hạng mục tùy biến.

- **Các thuộc tính:**
 - + `_id`: `ObjectId` (Khóa chính)
 - + `month`: `Number` (Tháng quản lý từ 1 đến 12)
 - + `year`: `Number` (Năm quản lý)
 - + `monthlyIncome`: `Number` (Thu nhập dự kiến)
 - + `monthlyBudget`: `Number` (Hạn mức chi tiêu tổng cho tháng)
 - + `monthlySavings`: `Number` (Mục tiêu tiết kiệm)
 - + `otherPlan`: `String` (Ghi chú kế hoạch khác)
 - + `categoryBudgets`: `Map<String, Number>` (Bản đồ phân bổ ngân sách riêng cho từng hạng mục)
 - + `categoryNamesMap`: `Map<String, String>` (Bản đồ lưu tên hạng mục)
 - + `customCategories`: `CustomCategory[]` (Mảng chứa các hạng mục tùy biến được nhúng trực tiếp)
 - + `createdAt`: `Date` / + `updatedAt`: `Date` (Thời gian hệ thống)

4. Lớp CustomCateGory (Hạng mục Tùy biến)

- **Ý nghĩa:** Là lớp thành phần được nhúng (Embedded/Sub-document) bên trong lớp `BudgetPlan` thông qua mối quan hệ Composition.

- **Các thuộc tính:**
 - + `id`: `String` (Mã hạng mục)
 - + `vi`: `String` / + `en`: `String` (Tên hiển thị song ngữ)
 - + `emoji`: `String` (Biểu tượng đại diện)
 - + `type`: `String` (Phân loại gốc: `expense` hoặc `income`)

2.4.3.2 Lớp Xử lý Nghiệp vụ

Phân tầng dịch vụ chịu trách nhiệm xử lý toàn bộ logic tính toán, truy vấn cơ sở dữ liệu và tương tác với các API trí tuệ nhân tạo bên thứ ba.

1. Lớp TransactionService

Các phương thức:

- + `getTransactions({type, date, month, year}): Transaction[]` (Lấy danh sách giao dịch theo bộ lọc)
- + `getThisMonthTransactions(): Transaction[]` (Lấy danh sách giao dịch tháng hiện tại)
- + `getLastMonthTransactions(): Transaction[]` (Lấy danh sách giao dịch tháng trước)
- + `createTransaction(data): Transaction` (Khởi tạo giao dịch mới)
- + `deleteTransaction(id): Transaction` (Xóa giao dịch theo mã định danh)
- + `computeSummary(): {totalIncome, totalExpense, balance}` (Tính toán tổng thu, tổng chi và số dư khả dụng)
- + `deleteAllTransactions(): void` (Xóa toàn bộ lịch sử giao dịch của người dùng)

2. Lớp ProfileService

Các phương thức:

- + `getProfile(): Profile` (Lấy thông tin hồ sơ hiện tại)
- + `updateProfile({name, email, monthlyBudget}): Profile` (Cập nhật thông tin hồ sơ và ngân sách)

3. Lớp AIService

Các phương thức:

- + `computeHealthScore(transactions, monthlyBudget): Number` (Tính toán điểm số sức khỏe tài chính từ 0 đến 100)
- + `computeInsights(thisMonth, lastMonth, monthlyBudget): InsightCard[]` (Phân tích đưa ra các thẻ gợi ý tài chính thông minh)
- + `computeSpendingBreakdown(transactions): SpendCategory[]` (Phân rã tỉ lệ chi tiêu chi tiết theo từng hạng mục)
- + `chatWithAI(query, financialContext, lang): String` (Xử lý hội thoại thời gian thực với Google Gemini API dựa trên ngữ cảnh tài chính)
- + `getAIData(monthlyBudget): AIOverview` (Tổng hợp cấu trúc dữ liệu tổng quan cho phân hệ AI)

2.4.3.3 Lớp Điều phối và Tiếp nhận yêu cầu

Lớp Controller chịu trách nhiệm định tuyến (Routing), tiếp nhận các yêu cầu HTTP Request (req) từ ứng dụng di động gửi lên, gọi các dịch vụ tương ứng xử lý và trả về phản hồi HTTP Response (res).

1. Lớp HomeController

Các phương thức:

- + getTransactions(req, res, next): void → GET /api/home/transactions
- + getSummary(req, res, next): void → GET /api/home/summary

2. Lớp AddTransactionController

Các phương thức:

- + createTransaction(req, res, next): void → POST /api/transactions
- + deleteTransaction(req, res, next): void → DELETE /api/transactions/:id

3. Lớp AIController

Các phương thức:

- - getCurrentMonthlyBudget(): void (Hàm trợ lý nội bộ lấy thông tin ngân sách tháng)
- + getOverview(req, res, next): void → GET /api/ai/overview
- + getHealthScore(req, res, next): void → GET /api/ai/health-score
- + getSpendingBreakdown(req, res, next): void → GET /api/ai/spending-breakdown
- + chat(req, res, next): void → POST /api/ai/chat

4. Lớp BudgetController

Các phương thức:

- + getBudgetPlan(req, res, next): void → GET /api/budget/plan
- + saveBudgetPlan(req, res, next): void → PUT /api/budget/plan

5. Lớp ChartController

Các phương thức:

- + getChartData(req, res, next): void → GET /api/charts/data

6. Lớp SettingController

Các phương thức:

- + `getProfile(req, res, next): void` → GET /api/settings/profile
- + `updateProfile(req, res, next): void` → PUT /api/settings/profile
- + `clearData(req, res, next): void` → DELETE /api/settings/data

2.4.3.4 Lớp Màn hình giao diện

Phân tầng này mô tả các thuộc tính trạng thái (State) quản lý cục bộ trên thiết bị và các phương thức xử lý sự kiện tương tác trực tiếp của người dùng tại ứng dụng di động.

1. Lớp HomeScreen (Màn hình chính)

- **Thuộc tính (State):** `transactions: Tx[], summary: Summary, loading: boolean, filter: TxType | "all", selectedDate: Date | null, showCal: boolean.`
- **Các phương thức:** `fetchData(), handleDelete(id), handleTypeFilter(type).`

2. Lớp AddTransaction (Màn hình Ghi chép)

- **Thuộc tính (State):** `txType: TxType, rawAmount: string, notes: string, selectedCat: string | null, monthlyBudget: string, catBudgets: Record<string, string>, customExpCats / customIncCats: Category[].`
- **Các phương thức:** `doSave(), handleSubmit(), triggerTxSaved(), triggerPlanSaved(), fetchBudgetData(), savePlan().`

3. Lớp AIAssistantScreen (Màn hình Trợ lý AI)

- **Thuộc tính (State):** `overview: AIOverview | null, messages: ChatMessage[], query: string, loading / aiLoading: boolean.`
- **Các phương thức:** `handleSend().`

4. Lớp SettingScreen (Màn hình Cài đặt)

- **Thuộc tính (State):** `name: string, email: string, editing: boolean, saving / loading: boolean.`
- **Các phương thức:** `handleSave(), handleClearData().`

5. Lớp ChartsScreen (Màn hình Biểu đồ phân tích)

- **Thuộc tính (State):** `period: "day" | "month" | "year", chartData: ChartData | null, loading: boolean.`

- **Các phương thức:** `fetchChart()`.

2.4.4 Các biểu đồ tuần tự

Nếu mô hình lớp (Class Diagram) cung cấp một cái nhìn tĩnh về cấu trúc hệ thống, thì biểu đồ tuần tự (Sequence Diagram) đóng vai trò mô hình hóa khía cạnh động của hệ thống. Phân hệ này mô tả chi tiết luồng truyền thông điệp (Message Passing) giữa các đối tượng theo trình tự thời gian, làm rõ cách thức các màn hình Frontend tương tác với các bộ điều khiển (Controllers), các dịch vụ nghiệp vụ (Services) và tầng dữ liệu (Models) để hoàn thành một tác vụ cụ thể.

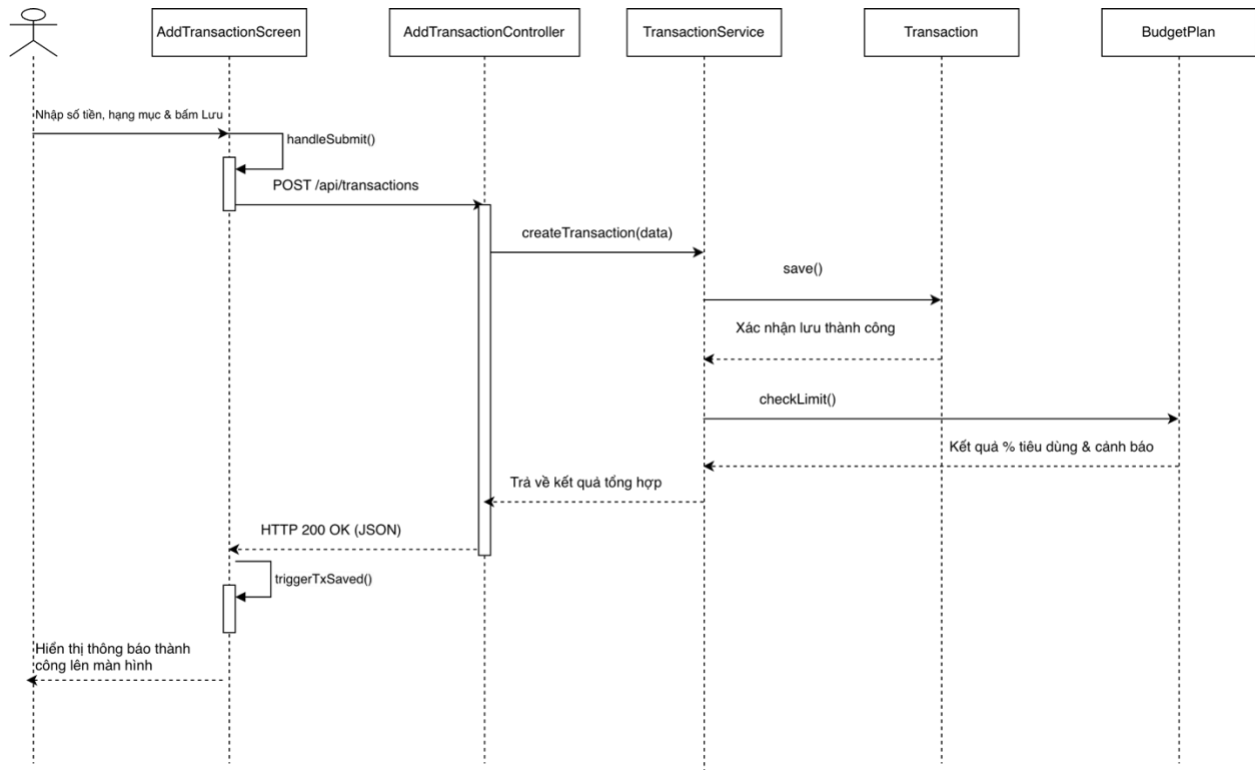
Để minh họa cho các quy trình nghiệp vụ cốt lõi của ứng dụng, hệ thống tiến hành xây dựng 3 biểu đồ tuần tự cho các chức năng trọng tâm bao gồm: Thêm mới giao dịch (kiểm tra ngân sách), Tương tác với Trợ lý AI và Tải thông tin tổng quan tại Trang chủ.

2.4.4.1 Biểu đồ tuần tự cho chức năng Thêm mới giao dịch (UC-01)

Biểu đồ này mô tả quy trình khi người dùng nhập một khoản thu/chi mới từ giao diện, hệ thống tiến hành kiểm tra ràng buộc ngân sách, ghi nhận vào cơ sở dữ liệu MongoDB và phản hồi trạng thái hiển thị cho người dùng.

- **Các đường đời (Lifelines) tham gia:**
 - Người dùng (Actor)
 - `AddTransactionScreen` (Frontend)
 - `AddTransactionController` (Backend Controller)
 - `TransactionService` (Backend Service)
 - `Transaction` (Model)
 - `BudgetPlan` (Model)
- **Trình tự các thông điệp:**
 1. Người dùng nhập dữ liệu số tiền, ghi chú, hạng mục và nhấn nút Lưu nghiệp vụ trên `AddTransactionScreen`.
 2. `AddTransactionScreen` thực hiện gọi hàm `handleSubmit()` nội bộ để đóng gói dữ liệu và gửi yêu cầu HTTP POST tới `AddTransactionController` qua Route `/api/transactions`.
 3. `AddTransactionController` tiếp nhận yêu cầu và gọi phương thức `createTransaction(data)` nằm trong `TransactionService`.
 4. `TransactionService` khởi tạo một đối tượng dữ liệu mới, gọi phương thức lưu trữ của Model `Transaction` để ghi đè vào cơ sở dữ liệu MongoDB Atlas.
 5. MongoDB xác nhận lưu thành công và trả về dữ liệu bản ghi cho `TransactionService`.
 6. `TransactionService` tiếp tục thực hiện truy vấn xuống Model `BudgetPlan` thông qua hàm tính toán nội bộ để kiểm tra xem giao dịch chi tiêu này có vượt quá hạn mức thiết lập hay không.
 7. `BudgetPlan` tính toán và trả về kết quả cấu hình (tỷ lệ phần trăm đã tiêu dùng).

8. TransactionService tổng hợp dữ liệu, trả kết quả kèm trạng thái cảnh báo tài chính về cho AddTransactionController.
9. AddTransactionController phản hồi kết quả dưới dạng chuỗi JSON dữ liệu về cho AddTransactionScreen.
10. AddTransactionScreen kích hoạt hàm triggerTxSaved(), cập nhật lại giao diện (đổi màu thanh trạng thái cảnh báo nếu cần) và thông báo thành công cho Người dùng.



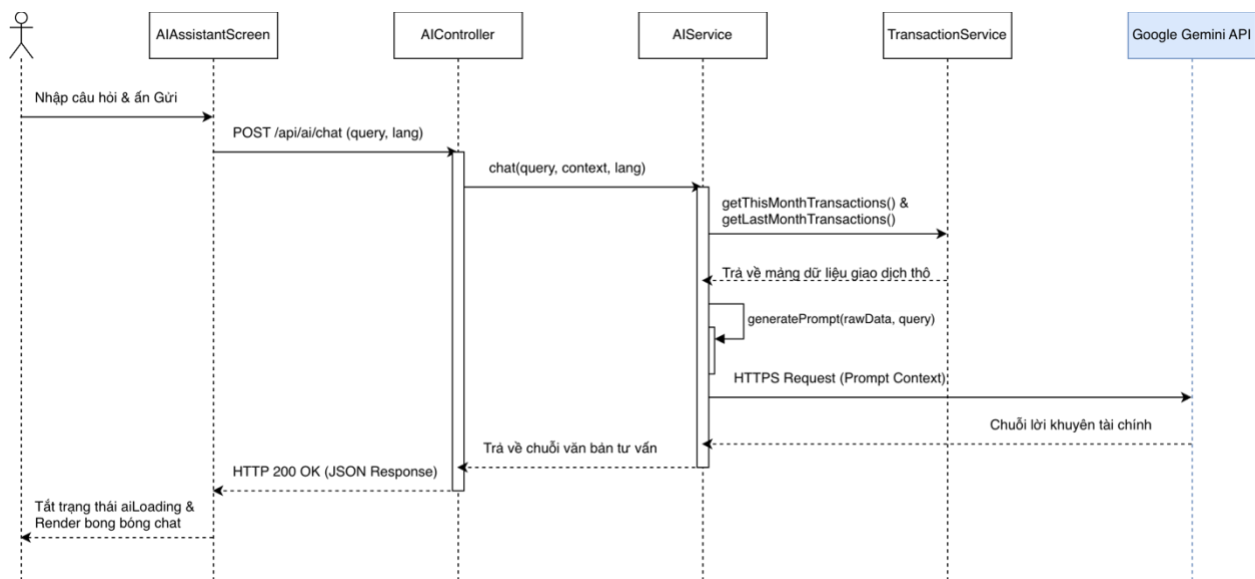
Sơ đồ 2.5. Biểu đồ tuần tự chức năng Thêm mới giao dịch.

2.4.4.2 Biểu đồ tuần tự cho chức năng Tương tác với Trợ lý AI (UC – 08)

Biểu đồ này thể hiện luồng xử lý thông minh khi người dùng gửi một câu hỏi yêu cầu phân tích tài chính. Hệ thống sẽ thu thập ngữ cảnh dữ liệu thực tế của người dùng để làm mốc dữ liệu (Prompt Context) gửi sang API của bên thứ ba.

- **Các đường đời (Lifelines) tham gia:**
 - Người dùng (Actor)
 - AIAssistantScreen (Frontend)
 - AIController (Backend Controller)
 - AIService (Backend Service)
 - TransactionService (Backend Service trung gian)
 - Google Gemini API (Thành phần liên kết ngoài qua HTTPS)
- **Trình tự các thông điệp:**

1. Người dùng nhập nội dung câu hỏi tại màn hình chat và nhấn nút gửi trên `AIAssistantScreen`.
2. `AIAssistantScreen` gọi phương thức xử lý sự kiện `handleSend()`, tạo yêu cầu `HTTP POST` gửi chuỗi truy vấn kèm ngôn ngữ lựa chọn tới `AIController` qua `Route /api/ai/chat`.
3. `AIController` tiếp nhận request, gọi hàm `chat(req, res, next)` và chuyển tiếp dữ liệu xuống hàm nghiệp vụ `chatWithAI(...)` của `AIService`.
4. `AIService` cần xây dựng ngữ cảnh tài chính thực tế nên tiến hành gọi hàm `getThisMonthTransactions()` và `getLastMonthTransactions()` từ `TransactionService`.
5. `TransactionService` truy vấn dữ liệu thô từ cơ sở dữ liệu và trả về danh sách mảng giao dịch của 2 tháng cho `AIService`.
6. `AIService` thực hiện hàm nội bộ `generatePrompt()` để kết hợp danh sách giao dịch, hạn mức ngân sách và câu hỏi của người dùng thành một gói dữ liệu `Prompt Context` chuẩn hóa.
7. `AIService` gửi một yêu cầu `HTTPS Request` (chứa `Prompt Context`) trực tiếp sang cổng `API` của `Google Gemini API`.
8. `Google Gemini API` tính toán, phân tích ngôn ngữ tự nhiên dựa trên dữ liệu tài chính nhận được và trả về chuỗi kết quả phản hồi (`Response String`) cho `AIService`.
9. `AIService` nhận kết quả văn bản tư vấn chuyên gia, trả ngược dữ liệu về cho `AIController`.
10. `AIController` đóng gói phản hồi thành cấu trúc `JSON` chuẩn và gửi trả về ứng dụng di động qua môi trường mạng.
11. `AIAssistantScreen` nhận mã kết quả, tắt trạng thái chờ `aiLoading` và render bong bóng chat chứa lời khuyên tài chính lên màn hình cho Người dùng đọc.



Sơ đồ 2.6. Biểu đồ tuần tự chức năng Tương tác với Trợ lý AI.

2.4.4.3 Biểu đồ tuần tự cho chức năng Tải dữ liệu tổng quan Trang chủ

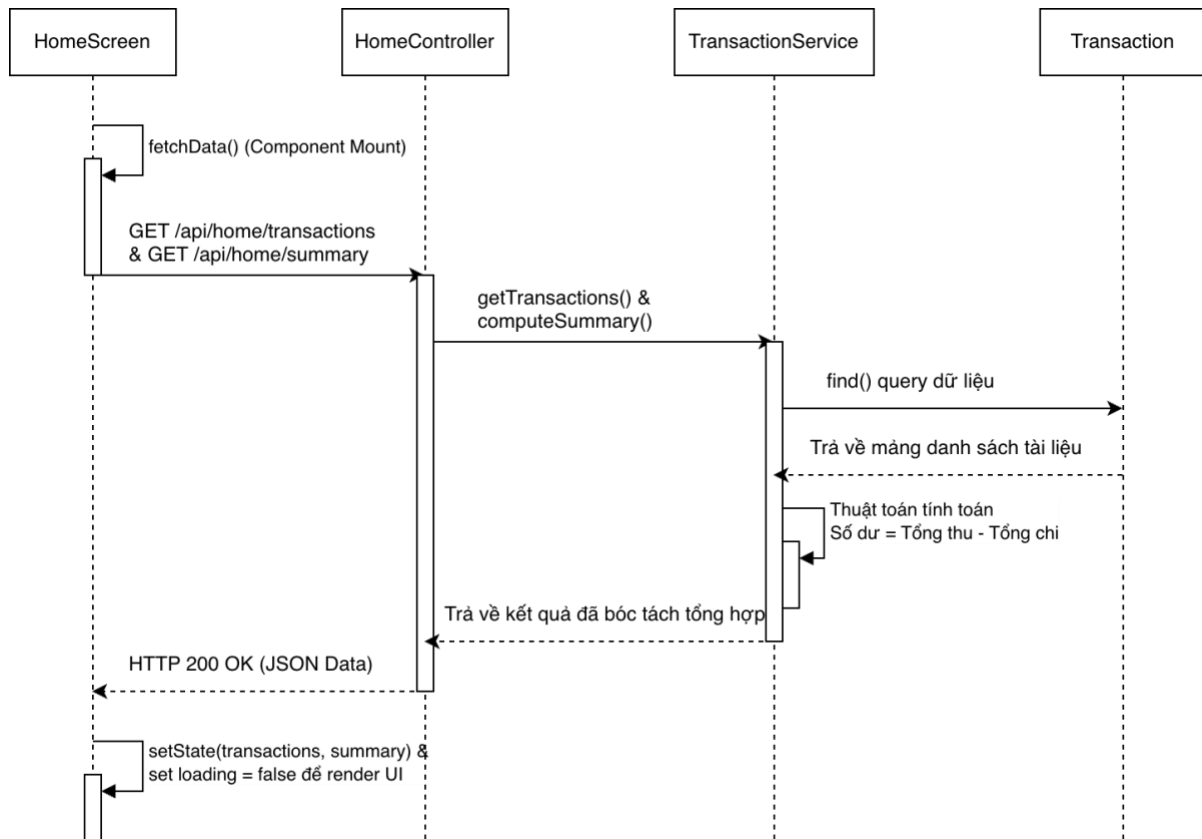
Biểu đồ này mô tả luồng hoạt động nền diễn ra ngay khi người dùng khởi động ứng dụng để hiển thị danh sách giao dịch gần đây và bảng tính toán số dư tổng thể.

- **Các đường đời (Lifelines) tham gia:**

- HomeScreen (Frontend)
- HomeController (Backend Controller)
- TransactionService (Backend Service)
- Transaction (Model)

- **Trình tự các thông điệp:**

1. Ngay khi vòng đời màn hình được kích hoạt (Component Mount), HomeScreen tự động gọi hàm `fetchData()` để lấy dữ liệu mới nhất.
2. HomeScreen gửi đồng thời các yêu cầu HTTP GET tới HomeController thông qua hai endpoint chính thức là `/api/home/transactions` và `/api/home/summary`.
3. HomeController điều phối luồng xử lý bằng cách gọi phương thức nghiệp vụ `getTransactions()` và `computeSummary()` nằm trong lớp `TransactionService`.
4. TransactionService thực hiện lệnh kết nối và truy vấn dữ liệu có bộ lọc từ Model Transaction.
5. Tầng dữ liệu của MongoDB trả về mảng các tài liệu giao dịch hiện có cho TransactionService.
6. TransactionService thực hiện thuật toán tính toán tổng số tiền thu, chi và số dư khả dụng dựa trên dữ liệu thô vừa lấy, sau đó trả kết quả tổng hợp về cho HomeController.
7. HomeController tập hợp cấu trúc dữ liệu, cấu hình mã trạng thái thành công (HTTP 200 OK) và phản hồi dữ liệu về cho Frontend.
8. HomeScreen tiếp nhận mảng dữ liệu, gán giá trị vào hai biến trạng thái cục bộ là `transactions` và `summary`, chuyển cờ `loading` sang giá trị `false` để hiển thị các thành phần giao diện trực quan lên thiết bị.



Sơ đồ 2.7. Biểu đồ tuần tự chức năng Tải dữ liệu Trang chủ.

2.4.5 Các màn hình giao diện người dùng

Giao diện người dùng (UI) là thành phần trực tiếp tương tác với người sử dụng, đóng vai trò chuyển hóa các yêu cầu chức năng thành các thành phần đồ họa trực quan. Hệ thống ứng dụng quản lý tài chính cá nhân được xây dựng dựa trên kiến trúc đa màn hình, đảm bảo tính tuần tự, đồng bộ dữ liệu theo thời gian thực và tối ưu hóa trải nghiệm người dùng (UX).

Dựa trên sơ đồ lớp thành phần giao diện và đặc tả ca sử dụng, cấu trúc các màn hình chức năng chính của ứng dụng được mô tả chi tiết dưới đây.

2.4.5.1 Màn hình Đăng nhập và Đăng ký

Đây là công xác thực bắt buộc để bảo mật thông tin và cá nhân hóa không gian lưu trữ dữ liệu tài chính của từng người dùng thông qua mô hình lớp `Profile`.

- **Màn hình Đăng nhập (Login Screen):**
 - **Mục đích:** Xác thực quyền truy cập của người dùng vào hệ thống bằng Email và Mật khẩu cá nhân.
 - **Các thành phần giao diện chính:** Ô nhập liệu Email (`email`), ô nhập Mật khẩu bảo mật mã hóa (`password`), nút lệnh "Đăng nhập", và liên kết chuyển hướng nhanh "Đăng ký tài khoản mới".

- **Luồng tương tác:** Người dùng điền thông tin tài khoản. Hệ thống tiến hành kiểm tra định dạng Email hợp lệ trực tiếp tại Client trước khi đóng gói gửi yêu cầu xác thực về phía Backend.
- **Màn hình Đăng ký (Register Screen):**
 - **Mục đích:** Cho phép người dùng mới thiết lập tài khoản định danh trên ứng dụng thông qua dịch vụ ProfileService.
 - **Các thành phần giao diện chính:** Trường nhập Họ và tên (name), trường nhập Email (email), trường Mật khẩu (password), và trường Xác nhận lại mật khẩu nhằm tránh sai sót khi nhập liệu.
 - **Ràng buộc kỹ thuật:** Mật khẩu nhập vào phải trùng khớp với trường xác nhận và đạt độ dài tiêu chuẩn hệ thống đề ra thì nút "Đăng ký" mới chuyển sang trạng thái kích hoạt (Active).

Hình 2.1. Giao diện Đăng nhập

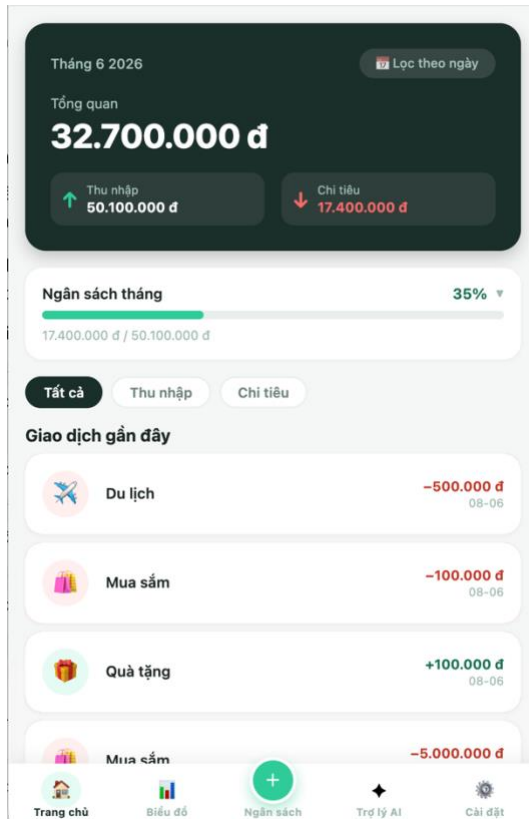
Hình 2.2. Giao diện Đăng ký

2.4.5.2 Màn hình Trang chủ

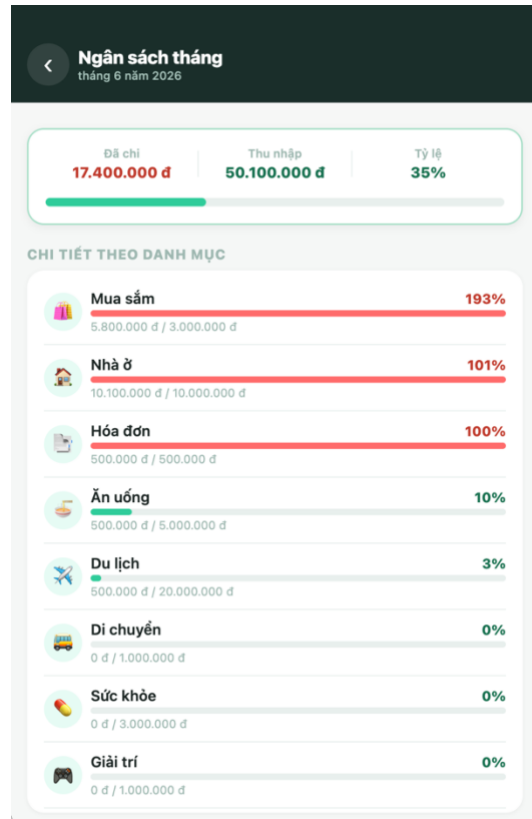
Màn hình hiển thị ngay sau khi người dùng xác thực tài khoản thành công, cung cấp một cái nhìn tổng quan, nhanh chóng về dòng tiền và các biến động số dư trong tháng.

- **Mục đích:** Hiển thị danh sách các giao dịch gần nhất và cung cấp các bộ lọc nhanh để người dùng theo dõi sức khỏe tài chính hiện tại.
- **Các biến trạng thái quản lý (State Fields):**
 - `transactions: Tx[]`: Mảng danh sách lưu trữ các bản ghi giao dịch thô tải về từ cơ sở dữ liệu.

- **summary:** Summary: Đối tượng chứa thông tin tổng hợp gồm tổng số tiền thu, chi và số dư khả dụng.
- **loading:** boolean: Cờ trạng thái hiển thị hiệu ứng chờ khi ứng dụng đang kết nối mạng để tải dữ liệu.
- **filter:** TxType | "all": Trạng thái bộ lọc, hỗ trợ phân loại danh sách hiển thị theo dạng "Thu nhập", "Chi phí" hoặc hiển thị tất cả.
- **selectedDate:** Date & **showCal:** boolean: Quản lý mốc thời gian xem dữ liệu và trạng thái ẩn/hiện của lịch cấu hình.
- **Các phương thức xử lý sự kiện (Methods):**
 - **fetchData():** Tự động kích hoạt khi gắn màn hình (Component Mount) để gọi dữ liệu tổng quan mới nhất.
 - **handleDelete(id):** Tiếp nhận sự kiện khi người dùng nhấn nút xóa một giao dịch trực tiếp từ danh sách.
 - **handleTypeFilter(type):** Xử lý thay đổi danh sách hiển thị tương ứng khi người dùng click đổi bộ lọc.



Hình 2.3. Giao diện Trang chủ



Hình 2.4. Giao diện Ngân sách tháng

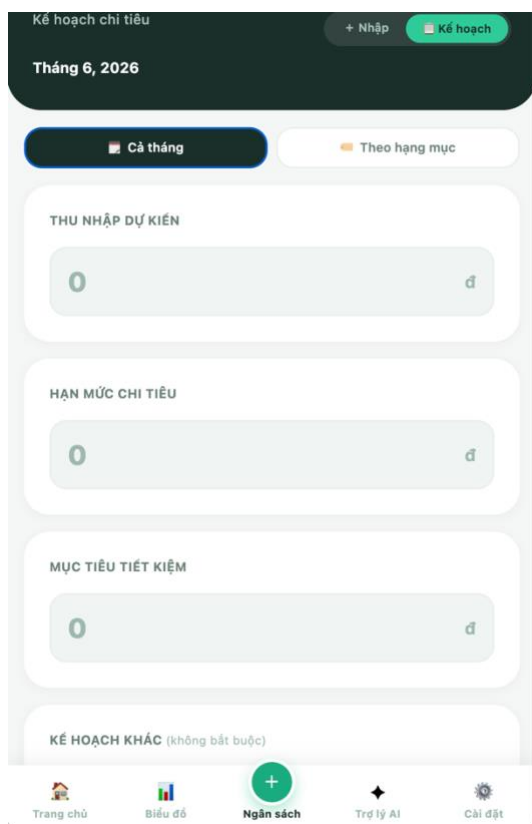
2.4.5.3 Màn hình Thêm mới và Quản lý giao dịch

Giao diện tích hợp đa năng xử lý biểu mẫu thu chi kiêm chức năng cấu hình hạn mức tài chính linh hoạt.

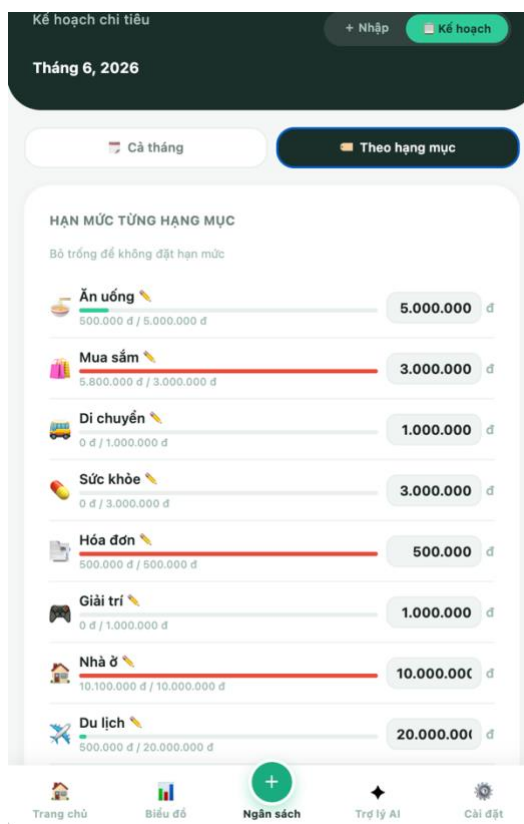
- **Mục đích:** Cho phép nhập liệu chi tiết các khoản thu nhập, chi phí phát sinh và thiết lập kế hoạch ngân sách giới hạn để phục vụ tính năng cảnh báo.
- **Các biến trạng thái quản lý (State Fields):**
 - txType: TxType: Phân loại giao dịch hiện tại là khoản thu (Income) hay khoản chi (Expense).
 - rawAmount: string & notes: string: Chuỗi ký tự số tiền thô nhập từ bàn phím và phần ghi chú đính kèm.
 - selectedCat: string: Mã danh mục hạng mục chi tiêu được người dùng lựa chọn từ danh sách thả xuống.
 - monthlyBudget: string & catBudgets: Record: Quản lý hạn mức tổng trong tháng và hạn mức phân chia riêng cho từng hạng mục cụ thể.
 - customExpCats & customIncCats: Mảng chứa các danh mục chi tiêu tự định nghĩa do người dùng tự bổ sung thêm.
- **Các phương thức xử lý sự kiện (Methods):**
 - handleSubmit() & doSave(): Đóng gói toàn bộ thông tin biểu mẫu, kiểm tra tính hợp lệ dữ liệu và tiến hành đẩy lệnh lưu xuống lớp dịch vụ Backend.
 - savePlan() & fetchBudgetData(): Xử lý lưu thiết lập ngân sách mới và thực hiện nạp dữ liệu cấu hình hạn mức cũ để hiển thị lên form.
 - triggerTxSaved() & triggerPlanSaved(): Các hàm phát sự kiện (Callback Event) làm tươi lại dữ liệu hệ thống ngay khi lưu thành công giao dịch hoặc ngân sách.

Hình 2.5. Giao diện Chi tiêu

Hình 2.6. Giao diện Thu nhập



Hình 2.7. Giao diện Lên kế hoạch theo Tháng



Hình 2.8. Giao diện Lên kế hoạch theo Hạng mục

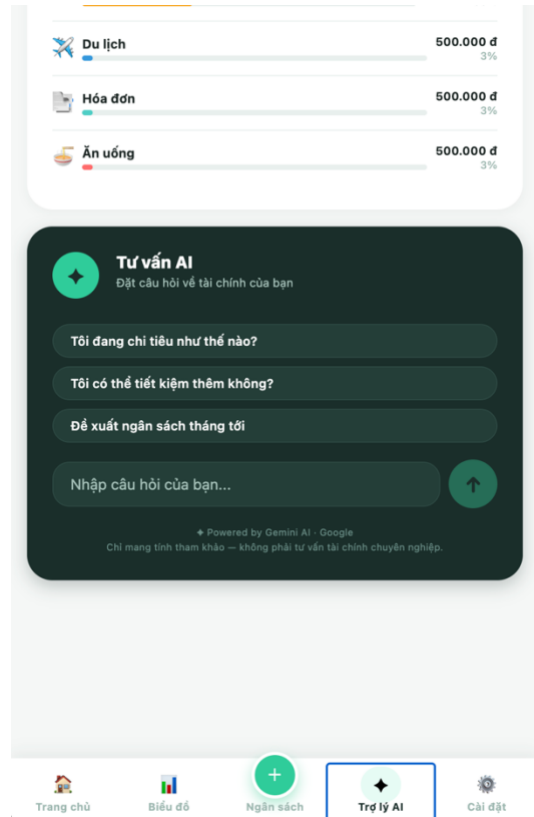
2.4.5.4 Màn hình Trợ lý ảo thông minh

Giao diện tương tác trò chuyện dạng Chatbot trực quan, kết nối người dùng với mô hình trí tuệ nhân tạo để nhận tư vấn tài chính.

- **Mục đích:** Cung cấp không gian tương tác bằng ngôn ngữ tự nhiên, cho phép đặt câu hỏi phân tích và nhận lời khuyên tối ưu ngân sách từ hệ thống AI chuyên gia.
- **Các biến trạng thái quản lý (State Fields):**
 - `overview: AIOverview`: Cấu trúc lưu trữ báo cáo tài chính tóm tắt tổng quan mà AI đọc để làm căn cứ phân tích dữ liệu.
 - `messages: ChatMessage[]`: Mảng danh sách các tin nhắn hội thoại bao gồm lời thoại của người dùng và phản hồi từ AI.
 - `query: string`: Chuỗi văn bản thô do người dùng nhập vào ô chat trước khi nhấn nút gửi.
 - `loading: boolean & aiLoading: boolean`: Quản lý cờ hiển thị tiến trình tải màn hình chat và trạng thái đợi phản hồi tính toán từ API Google Gemini.
- **Các phương thức xử lý sự kiện (Methods):**
 - `handleSend()`: Kích hoạt khi người dùng nhấn nút "Gửi", đẩy câu hỏi vào danh sách hiển thị cục bộ đồng thời kích hoạt API liên kết xử lý ngôn ngữ tự nhiên từ Backend.



Hình 2.9. Giao diện Tổng quan tài chính

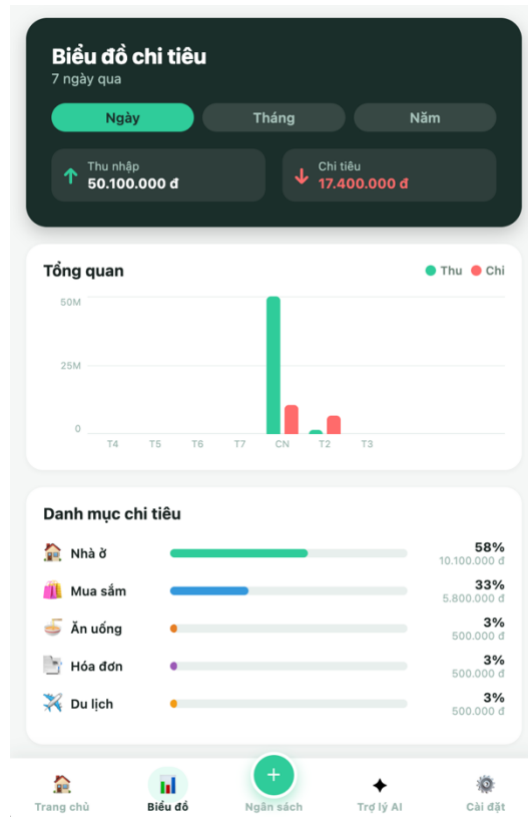


Hình 2.10. Giao diện Tư vấn AI

2.4.5.5 Màn hình Biểu đồ và Thống kê

Thành phần trực quan hóa dữ liệu dòng tiền, hỗ trợ người dùng có cái nhìn khoa học về các thói quen tiêu dùng cá nhân.

- **Mục đích:** Chuyển đổi các con số thô từ danh sách giao dịch thành biểu đồ hình quạt (Pie Chart) hoặc biểu đồ cột (Bar Chart) sinh động, bóc tách dòng tiền theo thời gian.
- **Các biến trạng thái quản lý (State Fields):**
 - `period: "day" | "month" | "year"`: Trạng thái xác định phạm vi quan sát phân tích dữ liệu theo Ngày, Tháng hoặc Năm.
 - `chartData: ChartData`: Gói cấu trúc dữ liệu đã bóc tách phân loại theo tỷ lệ %, sẵn sàng để nạp vào các thư viện vẽ biểu đồ của hệ thống Frontend.
 - `loading: boolean`: Quản lý trạng thái chờ trong khi thuật toán phân tích cơ sở dữ liệu đang chạy.
- **Các phương thức xử lý sự kiện (Methods):**
 - `fetchChart()`: Hàm thực hiện gửi yêu cầu lấy dữ liệu biểu đồ mới từ bộ điều khiển `ChartController` dựa trên khoảng thời gian (`period`) được chọn.

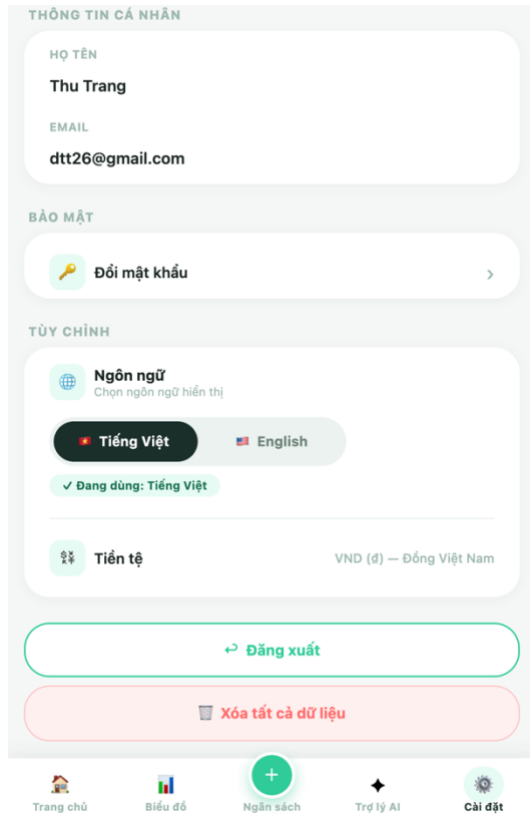


Hình 2.11. Giao diện Biểu đồ

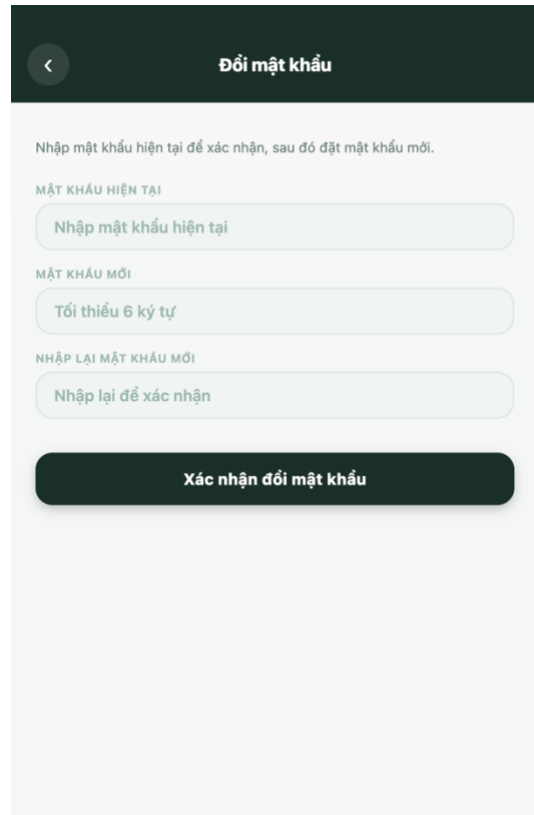
2.4.5.6 Màn hình Cài đặt và Cấu hình thông tin

Giao diện quản trị, cho phép người dùng tùy biến không gian làm việc và quản lý thông tin cốt lõi của tài khoản cá nhân.

- **Mục đích:** Cập nhật thông tin định danh cá nhân và cung cấp tính năng dọn dẹp, hoàn tác hệ sinh thái dữ liệu khi cần.
- **Các biến trạng thái quản lý (State Fields):**
 - `name: string & email: string`: Lưu trữ thông tin tên hiển thị và địa chỉ thư điện tử hiện tại của tài khoản.
 - `editing: boolean`: Cờ xác định trạng thái màn hình đang ở chế độ xem thông tin thông thường hay chế độ chỉnh sửa biểu mẫu.
 - `saving: boolean & loading: boolean`: Quản lý các trạng thái đợi trong tiến trình cập nhật dữ liệu tài khoản lên cơ sở dữ liệu đám mây.
- **Các phương thức xử lý sự kiện (Methods):**
 - `handleSave()`: Thực hiện lưu các thông tin chỉnh sửa mới về Họ tên của người dùng xuống lớp lưu trữ.
 - `handleClearData()`: Phương thức kích hoạt lệnh nguy hiểm, cho phép xóa toàn bộ lịch sử các giao dịch đã lưu của tài khoản này trên hệ thống MongoDB để làm sạch dữ liệu từ đầu.



Hình 2.12. Giao diện Cài đặt



Hình 2.13. Giao diện Tư vấn AI

Chương 3. Một số thành phần khác của dự án

3.1 Các vấn đề về đạo đức và làm việc chuyên nghiệp

Trong quy trình nghiên cứu, thiết kế và phát triển một hệ thống công nghệ thông tin – đặc biệt là ứng dụng quản lý tài chính tích hợp trí tuệ nhân tạo – khía cạnh đạo đức nghề nghiệp và tác phong làm việc chuyên nghiệp của người kỹ sư phần mềm luôn là kim chỉ nam hàng đầu. Các tiêu chuẩn này được cụ thể hóa qua ba yếu tố cốt lõi:

3.1.1 Đạo đức trong quản lý và bảo mật dữ liệu người dùng

- **Tính bảo mật và quyền riêng tư :** Dữ liệu tài chính bao gồm thu nhập, chi tiêu, thói quen sinh hoạt và thông tin định danh là những thông tin cực kỳ nhạy cảm của cá nhân. Hệ thống cam kết không chia sẻ dữ liệu thô của người dùng cho bất kỳ bên thứ ba nào khi chưa được phép. Việc thu thập thông tin chỉ phục vụ mục đích duy nhất là tính toán và tối ưu hóa ngân sách cá nhân cho chính họ.
- **Mã hóa và an toàn thông tin:** Tuân thủ nguyên tắc thiết kế an toàn, toàn bộ mật khẩu người dùng bắt buộc phải được băm (hash) bằng thuật toán mã hóa một chiều bảo mật (như `bcrypt`) trước khi lưu trữ vào cơ sở dữ liệu MongoDB. Mọi phiên làm việc giữa ứng dụng di động (Frontend) và hệ thống máy chủ (Backend) đều phải thông qua giao thức bảo mật mã hóa và sử dụng chuỗi định danh mã hóa (JSON Web Token - JWT) để chống lại các cuộc tấn công đánh cắp dữ liệu.

3.1.2 Đạo đức trong ứng dụng trí tuệ nhân tạo

- **Tính trung thực và độ tin cậy của thông tin:** Khi tích hợp Google Gemini API để đưa ra các lời khuyên tài chính, hệ thống được thiết lập bộ ngữ cảnh nghiêm ngặt (System Prompt Context). Điều này nhằm hạn chế tối đa hiện tượng "ảo tưởng" (hallucination) của mô hình ngôn ngữ lớn, đảm bảo các phân tích đưa ra là trung thực, mang tính xây dựng và bám sát vào số liệu thực tế thô (`rawData`) của người dùng.
- **Tính minh bạch :** Ứng dụng phân định rõ ràng giữa "lời khuyên mang tính tham khảo từ AI" và "quyết định tài chính thực tế của người dùng". Trợ lý ảo đóng vai trò hỗ trợ, không can thiệp, áp đặt hoặc tự động thực hiện các giao dịch làm thay đổi số dư tài khoản của người sử dụng.

3.1.3 Tác phong làm việc chuyên nghiệp và sở hữu trí tuệ

- **Tuân thủ bản quyền nguồn mở:** Trong quá trình phát triển ứng dụng, các thư viện nguồn mở sử dụng trong hệ sinh thái React Native, Express.js và Mongoose đều được kiểm tra giấy phép (License) kỹ càng, đảm bảo tuân thủ đúng quy định sử dụng phần mềm cộng đồng.
- **Tôn trọng sở hữu trí tuệ:** Tuyệt đối không sao chép mã nguồn, cấu trúc hệ thống của các sản phẩm thương mại khác mà không có sự thừa nhận hoặc cải tiến mang dấu ấn cá nhân. Mọi tài liệu tham khảo, các thuật toán phân tích số dư, tổng thu - tổng chi đều được trích dẫn nguồn rõ ràng trong tài liệu đề án.

3.2 Tác động xã hội

Ứng dụng quản lý tài chính cá nhân tích hợp AI không chỉ dừng lại là một công cụ công nghệ thuần túy, mà còn hướng tới việc giải quyết các bài toán thực tiễn trong đời sống, mang lại nhiều giá trị tích cực cho cộng đồng.

3.2.1 Nâng cao nhận thức và năng lực tư duy tài chính cá nhân

- **Xây dựng thói quen tích lũy:** Khó khăn lớn nhất của đa số người trẻ hiện nay là kiểm soát dòng tiền chi tiêu hàng ngày. Ứng dụng giúp đơn giản hóa việc ghi chép giao dịch, giúp người dùng nhận thức được tiền của mình đang "chảy" đi đâu thông qua các biểu đồ trực quan hóa dữ liệu trực quan.
- **Ngăn ngừa rủi ro nợ nần:** Bằng tính năng thiết lập hạn mức ngân sách (`checkLimit()`), ứng dụng phát đi cảnh báo chủ động khi chi tiêu chạm ngưỡng. Điều này giúp người dùng điều chỉnh hành vi mua sắm kịp thời, thúc đẩy lối sống tiêu dùng thông minh và bền vững.

3.2.2 Bình đẳng hóa cơ hội tiếp cận tư vấn chuyên gia nhờ công nghệ

- Trước đây, việc có một cố vấn tài chính riêng để phân tích thu chi và đưa ra lời khuyên tối ưu dòng tiền hàng tháng là đặc quyền tiêu tốn nhiều chi phí. Với việc tích hợp thành công mô hình ngôn ngữ lớn thông qua Google Gemini API, ứng dụng đã cung cấp một "trợ lý tài chính ảo" hoạt động 24/7 hoàn toàn miễn phí cho tất cả mọi người, giúp rút ngắn khoảng cách tiếp cận tri thức quản lý tài chính giữa các tầng lớp xã hội khác nhau.

3.2.3 Tác động tích cực đến tâm lý người tiêu dùng

- Việc mất kiểm soát tài chính thường dẫn đến áp lực tâm lý và căng thẳng kéo dài. Ứng dụng mang lại cảm giác an tâm khi dòng tiền được kiểm soát rõ ràng, khoa học, giúp người dùng tự tin hơn trong các quyết định chi tiêu lớn nhỏ hay các kế hoạch đầu tư tương lai.

3.3 Kế hoạch cho kiến thức mới và chiến lược học tập

Quá trình hoàn thiện dự án này là một cơ hội lớn để em có thể nghiên cứu và thực nghiệm các công nghệ hiện đại, đồng thời mở ra những hướng đi mới cần tiếp tục trau dồi trong tương lai.

3.3.1 Những kiến thức mới đã tiếp thu thành công qua dự án

- **Phát triển ứng dụng Full-Stack:** Giúp em làm chủ quy trình xây dựng ứng dụng di động đa nền tảng bằng React Native kết hợp với hệ thống API RESTful hướng dịch vụ bằng Node.js/Express và cơ sở dữ liệu linh hoạt NoSQL MongoDB.
- **Kỹ nghệ Prompt (Prompt Engineering):** Học được cách thiết kế câu lệnh điều hướng hệ thống, xử lý định dạng dữ liệu đầu vào và đầu ra để kết nối mượt mà giữa ứng dụng phần mềm truyền thống với trí tuệ nhân tạo (LLM).

- **Tư duy thiết kế hệ thống tuần tự:** Cho em nắm vững cách chuyển đổi từ các yêu cầu nghiệp vụ thực tế thành các sơ đồ kiến trúc chuẩn UML (Sơ đồ ca sử dụng, sơ đồ lớp, sơ đồ tuần tự), giúp tối ưu hóa logic viết mã nguồn.

3.3.2 Những hạn chế hiện tại của hệ thống cần khắc phục

- **Tính cá nhân hóa sâu của AI:** Trợ lý ảo hiện tại chỉ phân tích dựa trên tập dữ liệu tĩnh gửi lên tại thời điểm chat, chưa có khả năng ghi nhớ dài hạn xu hướng chi tiêu qua nhiều tháng của một cá nhân cụ thể.
- **Khả năng hoạt động ngoại tuyến:** Ứng dụng phụ thuộc hoàn toàn vào kết nối mạng Internet ổn định để đồng bộ API dữ liệu và truy vấn AI. Nếu mất mạng, người dùng không thể ghi chép nhanh giao dịch.
- **Cơ chế bảo mật ở mức cơ bản, chưa tối ưu cho dữ liệu tài chính chuyên sâu:**
 - *Thiếu xác thực sinh trắc học (Biometric Authentication):* Hệ thống hiện tại mới chỉ dừng lại ở việc xác thực tài khoản qua Email/Mật khẩu truyền thống và lưu Token phiên làm việc. Ứng dụng chưa được tích hợp các giải pháp quét vân tay (Touch ID) hoặc nhận diện khuôn mặt (Face ID) trực tiếp trên thiết bị di động. Điều này làm giảm tính tiện dụng và tăng nguy cơ bị nhìn trộm mật khẩu khi người dùng thao tác ở nơi công cộng.
 - *Chưa mã hóa dữ liệu tĩnh chuyên sâu (Data-at-Rest Encryption):* Mặc dù mật khẩu tài khoản đã được băm bảo mật bằng thuật toán `bcrypt` ở phía Server, nhưng các thông tin nhạy cảm khác như số dư tài khoản, chi tiết lịch sử thu chi và nội dung chat với AI lưu trữ trong cơ sở dữ liệu MongoDB hoặc bộ nhớ đệm cục bộ của điện thoại vẫn ở dạng văn bản thô, chưa được mã hóa cấp trường (Field-level encryption). Nếu hệ thống hoặc thiết bị bị chiếm quyền điều khiển, các thông tin này có nguy cơ bị rò rỉ.
 - *Rủi ro an toàn thông tin khi chia sẻ với bên thứ ba (AI API):* Việc gửi dữ liệu giao dịch thô lên mô hình Google Gemini API thông qua môi trường Internet, dù có sử dụng giao thức HTTPS, vẫn tiềm ẩn các nguy cơ về an toàn thông tin nếu chính sách bảo mật dữ liệu của bên cung cấp dịch vụ AI thay đổi, hoặc khi tài khoản API Key bị lộ lọt do cấu hình môi trường (`.env`) chưa được bảo vệ bằng các giải pháp quản lý khóa chuyên nghiệp (như Vault).
 - *Thiếu cơ chế xác thực hai lớp (MFA) và kiểm soát tần suất tấn công:* Hệ thống chưa có tính năng gửi mã OTP qua Email/SMS để xác nhận khi người dùng thực hiện các tác vụ nguy hiểm (chẳng hạn như lệnh xóa toàn bộ dữ liệu lịch sử tài khoản tại mục Cài đặt). Đồng thời, hệ thống Backend chưa cấu hình các lớp phòng thủ nâng cao như chống tấn công dò mã (Brute-force protection) hay giới hạn tần suất gửi yêu cầu liên tục (Rate-limiting) để ngăn chặn các cuộc tấn công từ chối dịch vụ (DoS).

Chương 4. Kết luận

Sau một thời gian tập trung nghiên cứu, thiết kế và phát triển, dự án xây dựng ứng dụng quản lý tài chính cá nhân tích hợp trí tuệ nhân tạo đã hoàn thành toàn bộ các mục tiêu đề ra ban đầu. Dưới đây là nội dung tổng kết, tự đánh giá và định hướng phát triển kế tiếp của sản phẩm.

4.1 Kết quả đạt được của dự án

Dự án đã hiện thực hóa thành công từ ý tưởng, sơ đồ kiến trúc chuẩn UML vào một sản phẩm phần mềm thực tế, hoàn thiện trọn vẹn ở cả ba lớp kiến trúc (Frontend, Backend, Database):

4.1.1 Về mặt kiến trúc và kỹ năng kỹ thuật

- **Làm chủ quy trình phát triển sản phẩm (Full-Stack):** Nắm vững tư duy lập trình hệ thống, kiểm soát tốt luồng dữ liệu bất đồng bộ giữa ứng dụng di động đa nền tảng (React Native) và hệ thống máy chủ API RESTful (Node.js/Express.js).
- **Ứng dụng công nghệ mới:** Tiếp cận và tích hợp thành công các mô hình ngôn ngữ lớn (LLM) thông qua việc kết nối API Google Gemini; làm chủ kỹ nghệ thiết kế câu lệnh ngữ cảnh (Prompt Engineering) để đưa AI vào giải quyết bài toán thực tế.
- **Tư duy phân tích thiết kế hệ thống:** Biết cách chuyên hóa hệ thống yêu cầu chức năng phức tạp thành các thực thể dữ liệu rõ ràng, minh bạch thông qua các biểu đồ tuần tự và sơ đồ lớp chi tiết.

4.1.2 Về mặt sản phẩm công nghệ hoàn thiện

Hệ thống ứng dụng đã vận hành ổn định và đạt được các kết quả cụ thể sau:

- **Hệ thống xác thực an toàn:** Xây dựng hoàn chỉnh tính năng Đăng nhập và Đăng ký tài khoản thông qua Email và Mật khẩu. Mật khẩu được băm mã hóa một chiều bằng `bcrypt` và phiên làm việc được quản lý bảo mật bằng cơ chế JSON Web Token (JWT).
- **Quản lý nghiệp vụ tài chính cốt lõi:**
 - Ghi chép, phân loại và lưu trữ chính xác các giao dịch Thu nhập/Chi phí theo thời gian thực.
 - Tự động tính toán số dư tổng quan ($Số dư = Tổng thu - Tổng chi$) hiển thị trực quan lên màn hình chính.
 - Thiết lập hạn mức ngân sách chi tiêu linh hoạt, tự động kích hoạt cơ chế kiểm tra và phát cảnh báo vượt ngưỡng (`checkLimit`) kịp thời cho người dùng.
- **Trợ lý ảo thông minh hoạt động hiệu quả:** Kết nối thành công với mô hình trí tuệ nhân tạo để đọc hiểu báo cáo tài chính thô của người dùng, từ đó đưa ra các chuỗi văn bản tư vấn tối ưu ngân sách bằng văn phong tự nhiên.
- **Giao diện trực quan và đồng bộ (UI/UX):** Hoàn thiện bộ 6 màn hình chức năng chính (Login/Register, HomeScreen, AddTransactionScreen, AIAssistantScreen, ChartsScreen, SettingsScreen) đáp ứng tính scannable, mượt mà và trực quan hóa số liệu qua hệ thống biểu đồ tròn/biểu đồ cột sinh động.

4.2 Hạn chế còn tồn tại

Dù đạt được những kết quả rất tích cực, hệ thống vẫn tồn tại một số điểm hạn chế nhất định cần được thẳng thắn nhìn nhận để cải thiện:

- **Tốc độ phản hồi của hệ thống AI có đôi lúc còn chậm (Latency):** Do ứng dụng gọi truy vấn trực tiếp đến API Google Gemini từ xa thông qua môi trường Internet, thời gian xử lý và trả kết quả của mô hình ngôn ngữ lớn (LLM) phụ thuộc nhiều vào băng thông mạng và tải lượng từ máy chủ Google. Điều này dẫn đến việc người dùng đôi khi phải chờ đợi từ 3 đến 5 giây để nhận được câu trả lời trong khung chat, làm giảm tính liền mạch của trải nghiệm UX.
- **Độ chính xác của hệ thống AI chưa đạt mức tuyệt đối:** Do bản chất của các mô hình ngôn ngữ lớn vẫn có tỷ lệ gặp hiện tượng "ảo tưởng" (hallucination), trợ lý ảo đôi lúc còn đưa ra các nhận định chung chung, chưa hoàn toàn bám sát hoặc tính toán sai lệch một số con số nhỏ khi xử lý các tập dữ liệu tài chính thô (`rawData`) quá phức tạp hoặc có nhiều danh mục đan xen. Hệ thống chưa thể đảm bảo độ chính xác 100% trong mọi tình huống hội thoại phức tạp.
- **Bảo mật hệ thống ở mức cơ bản:** Ứng dụng chưa được tích hợp các giải pháp xác thực sinh trắc học hiện đại (Face ID, Touch ID) trực tiếp trên thiết bị di động, cũng như chưa có cơ chế xác thực hai lớp (MFA) đối với các tác vụ nguy hiểm như xóa sạch dữ liệu lịch sử.
- **Phụ thuộc vào môi trường mạng (Internet):** Ứng dụng vận hành hoàn toàn phụ thuộc vào kết nối trực tuyến. Hệ thống chưa hỗ trợ cơ chế lưu đệm (caching) cục bộ để người dùng có thể ghi chép nhanh giao dịch khi mất mạng.
- **Thiếu bộ nhớ dài hạn:** Trợ lý ảo AI mới chỉ xử lý dữ liệu dựa trên tập ngữ cảnh tĩnh tại một thời điểm cụ thể, chưa có khả năng lưu trữ bộ nhớ dài hạn qua nhiều tháng để tự động nhận diện và học sâu hành vi tiêu dùng mang tính chu kỳ của từng cá nhân.

4.3 Hướng phát triển trong tương lai

- **Tối ưu hóa và nâng cao năng lực lõi AI:**
 - Triển khai cơ chế **Streaming dữ liệu (Server-Sent Events)** cho API của Trợ lý ảo để phản hồi văn bản hiển thị ra màn hình theo từng từ ngay lập tức, giải quyết triệt để cảm giác chờ đợi do tốc độ phản hồi chậm.
 - Nghiên cứu áp dụng kỹ thuật **RAG (Retrieval-Augmented Generation)** và tinh chỉnh câu lệnh hệ thống (System Prompt) để giới hạn chặt chẽ phạm vi tính toán toán học, giúp nâng cao độ chính xác, loại bỏ các câu trả lời sai lệch thông tin tài chính.
- **Xây dựng chế độ hoạt động ngoại tuyến (Offline Mode):** Triển khai cơ sở dữ liệu nhúng cục bộ (như SQLite hoặc Realm) ngay trên thiết bị di động. Cho phép người dùng ghi chép chi tiêu mọi lúc mọi nơi, sau đó hệ thống sẽ tự động thực hiện thuật toán đồng bộ (Sync) đồng nhất dữ liệu về máy chủ MongoDB khi thiết bị có kết nối mạng trở lại.

- **Nâng cấp toàn diện kiến trúc bảo mật:** Tích hợp bộ thư viện sinh trắc học của hệ điều hành di động cho phép đăng nhập nhanh bằng vân tay/khuôn mặt; xây dựng tính năng gửi mã OTP xác nhận qua Email và thực hiện mã hóa nâng cao cấp trường đối với dữ liệu lịch sử thu chi lưu trên đám mây.
- **Chuẩn hóa quy trình vận hành hạ tầng (DevOps):** Thực hiện đóng gói toàn bộ hệ thống Backend và Database bằng công nghệ Docker Container; cấu hình chu trình tích hợp và triển khai tự động (CI/CD) lên các hạ tầng điện toán đám mây như AWS hoặc Google Cloud để đảm bảo khả năng chịu tải, sẵn sàng mở rộng quy mô người dùng lớn trong tương lai.

Chương 5. Tài liệu tham khảo

- Nguyễn Văn Ba, *Phân tích thiết kế hệ thống hướng đối tượng bằng UML*, Nhà xuất bản Đại học Quốc gia Hà Nội, 2015.
- Đỗ Trung Tuấn, *Cơ sở dữ liệu*, Nhà xuất bản Đại học Quốc gia Hà Nội, 2018.
- Nguyễn Khánh Hùng, *Lập trình ứng dụng di động đa nền tảng với React Native*, Nhà xuất bản Bách Khoa Hà Nội, 2021.
- Meta Platforms Inc, “React Native Documentation”, <https://reactnative.dev>.
- OpenJS Foundation, “Node.js Documentation & Express.js Guide”, <https://nodejs.org> và <https://expressjs.com>.
- Các bài viết, tài liệu kỹ thuật và hướng dẫn phát triển ứng dụng Full-Stack, RESTful API và tích hợp Trí tuệ nhân tạo (LLM) trên Internet